

Graph neural networks for network intrusion detection systems: a comprehensive survey, taxonomy, and empirical evaluation

Mohammad Reza Ismaeli Taba · Hadi Zare[✉] · Saeid Rakhshani

Received: 15 December 2025 / Accepted: 10 August 2026

© The Author(s) 2026

Abstract

Graph neural networks (GNNs) have emerged as a transformative paradigm for intrusion detection systems (IDSs), addressing limitations of traditional flat-data approaches that fail to capture complex network structures and attack patterns. This survey systematically reviews GNN-driven IDS solutions and introduces a comprehensive taxonomy spanning problem domain specialization, graph construction, feature encoding, learning paradigms, and evaluation methodologies. We analyze existing literature across seven key dimensions, examining node- and edge-level detection tasks, static versus dynamic graph models, and supervised, semi-supervised, and unsupervised learning settings. The review critically evaluates well-known GNN architectures across diverse network environments such as IoT, smart grids, and vehicular networks. To strengthen the quantitative component, we conduct a leakage-controlled study using the UNSW-NB15 dataset and perform a comprehensive performance evaluation of ten representative IDS baselines. Empirical results reveal that graph-based IDS models offer stronger practical advantages in detection rate stability, inference efficiency, and near-real-time applicability compared to conventional tree-based models. These findings suggest that the value of GNN-based IDS should be assessed not solely through aggregate accuracy but also through operational metrics such as detection rate, running time, scalability, and graph construction feasibility.

Keywords Graph neural networks · Network intrusion detection · Graph-based security analytics · Cyber-security · IDS taxonomy · Real-time intrusion detection

1 Introduction

This section introduces the motivation, scope, and positioning of the survey. It first outlines the role of intrusion detection systems in modern network security and explains why conventional signature-based, anomaly-based, machine-learning, and deep-learning approaches are important but limited baselines. It then motivates the use of graph-based and GNN-based intrusion detection by showing how network traffic, hosts, flows, alerts, and system events naturally form relational structures. Finally, the section summarizes the main contributions of the survey and clarifies how the paper differs from prior GNN-IDS reviews.

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.

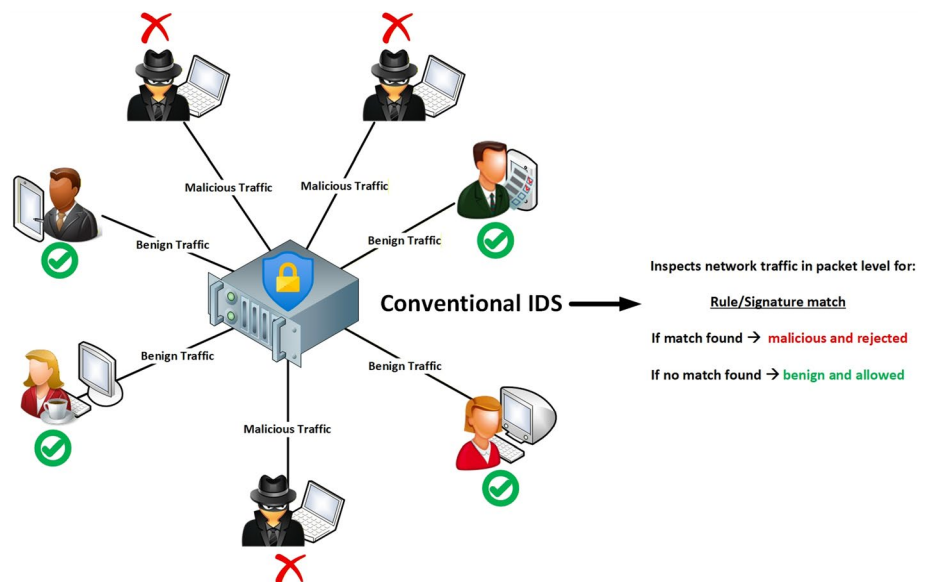


1.1 Intrusion detection systems (IDS)

An intrusion detection system (IDS) is a security mechanism that continuously monitors network or host activities to identify unauthorized or malicious behavior (Ahanger et al. 2025). In particular, a Network-based IDS (NIDS) is placed at network entry points (such as firewalls or monitoring ports) and inspects the packet flows across many hosts (Khraisat et al. 2019). By examining packet headers and payloads, a NIDS can detect suspicious external activities -such as port scans, protocol anomalies, or malicious payloads- and alert before they reach their targets (Khraisat et al. 2019). A Host-based IDS (HIDS), in contrast, resides on each machine and analyzes local logs or process events to detect intrusions on that host (Khraisat et al. 2019). NIDS offers broad network-level visibility, while HIDS provides fine-grained per-host insight. However, the focus of this paper is mainly restricted to network-based intrusion detection systems (NIDS). Accordingly, unless otherwise noted, the abbreviation IDS is used to designate network-based systems. Conventional IDS methods are generally grouped into signature-based and anomaly-based detection. In signature-based IDS, the system maintains a database of known attack patterns or rules. Each incoming packet or event is matched against this signature database, and a match triggers an alert. Signature systems (e.g., Snort) recognize known attacks with high accuracy and low false-alarm rates, but by design they cannot detect novel or obfuscated threats. On the other hand, anomaly-based IDS, first learns a model of "normal" behavior (using statistics, clustering or machine learning) and flags deviations from this baseline as potential intrusions. The key advantage is that anomaly detection can identify previously unseen attacks (zero-days) by spotting behavior that diverges from the norm. The trade-off is that anomaly systems often produce higher false-positive rates (Khraisat et al. 2019), since legitimate but rare variations may appear anomalous. In practice, anomaly IDS must be carefully tuned and regularly updated to balance sensitivity and false alarms. The schematic flow diagram in Fig. 1 for the conventional Intrusion Detection System (IDS) illustrates the sequential process for detecting network intrusions using methods. The diagram begins with a data collection phase, where network traffic (e.g., packets, logs) is gathered from sources like routers or sensors. The data then flows into a detection engine, where based on matching specified rules or signatures, the system generates alerts for identified intrusions, leading to response actions (e.g., logging or blocking traffic).

Despite decades of research (Zhong et al. 2024), IDS faces several key challenges today. Contemporary networks generate high-dimensional, high-volume data, as millions of packets and flows, each with many features (e.g. IP headers, ports, payload bytes, connection flows), must be processed (Pramanick et al. 2025). Extracting salient features of the big data streams for real-time detection systems is difficult. Meanwhile, cyber threats

Fig. 1 Conceptual depiction of a conventional IDS, in which packet- and flow-level traffic collected from multiple hosts is inspected by a centralized detection engine that applies predefined rules or signatures to classify activity as benign (permitted) or malicious (flagged and rejected), thereby enabling rule-driven alerting and basic response actions



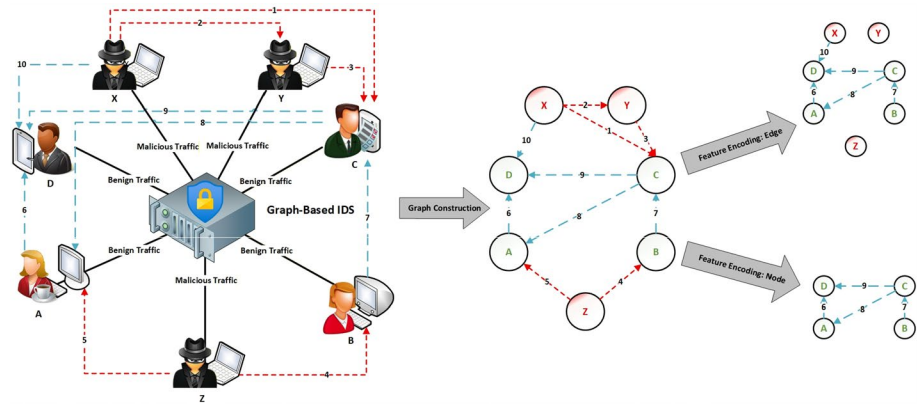
evolve constantly: attackers use polymorphic malware, stealthy exploits, and complex multi-stage attacks. As Skarding et al. (2021) cleverly modified exploits (Khraisat et al. 2019). Network communications naturally form complex graphs (Zade 2024). Specifically, a network can be represented as a graph where nodes are hosts or IP addresses and edges are communication flows between them (possibly labeled with features like traffic volume or protocol type). In graph-based viewpoint, multi-step attacks appear as distinctive subgraph patterns: a port scan might manifest as one node connected via edges to many others, and a botnet as a cluster of devices talking to the same malicious server. Graph-based models capture such relational patterns that flat, packet-level features miss. As Bilot et al. (2023b) observe, intrusion events have been represented via flow or provenance graphs for years, because “the structural patterns of attacks” become apparent in graph form Bilot et al. (2023b). Similarly, Ahanger et al. (2025) stated that IoT traffic “exhibits graph-like structure”, motivating graph-based analysis in IDS (Ahanger et al. 2025). By modeling traffic as a graph, an IDS can exploit this topology to better recognize distributed or correlated threats. Traditional machine-learning and deep-learning IDS models are important baselines because they are often easier to train, faster to deploy, and more mature in operational pipelines. Random-forest classifiers, SVM-family detectors, CNNs, and RNNs can perform well when attacks are separable using compact flow statistics or short temporal signatures. Recent traffic-analysis studies further show that carefully designed non-graph representations remain competitive in specific settings; for example, frequency-domain features can support robust malicious-traffic detection, while sample-efficient cross-modality fusion can reduce dependence on large labeled attack sets (Fu et al. 2023, 2025).

These approaches clarify the role of GNN-based IDS. Conventional traffic-analysis models usually treat each flow, packet sequence, or engineered feature vector as an isolated sample, which limits their ability to represent shared hosts, repeated communication patterns, lateral movement, multi-stage campaigns, and correlated botnet behavior. GNN-based IDS reframes the detection task around relations among flows, hosts, ports, users, devices, alerts, or system events. Graph message passing becomes particularly useful when malicious behavior is distributed across multiple entities and cannot be fully captured by per-flow features alone.

1.2 Graph neural networks for IDS

Graph neural networks (GNNs) are deep learning models specifically designed to learn from graph-structured data. A GNN operates by iterative message-passing: each node in the graph maintains a feature vector (embedding), and in each layer the node aggregates feature information from its neighbors and updates its own embedding. After several layers, a node’s final embedding encodes information from its multi-hop neighborhood and the overall graph topology (Ahanger et al. 2025). In the IDS context, it means a node’s representation reflects both its own traffic attributes and the patterns of traffic around it in the network. This capability makes GNNs well-suited for intrusion detection applications. A GNN-based IDS can jointly consider individual traffic features and network topology when making predictions. For example, if one device in the graph begins sending malicious packets, a GNN can propagate an increased anomaly score along its edges, reflecting that neighboring hosts might also be suspicious. Conversely, normal regions of the network reinforce benign behavior. Crucially, GNNs can learn feature representations from the raw graph without requiring manual feature engineering (Bilot et al. 2023b). Recent studies show that GNN-based IDS often outperform traditional ML models on standard benchmarks. Indeed, GNNs have “gained prominence as a powerful technique for enhancing IDS” due to their ability to model complex inter-node relationships (Ahanger et al. 2025). By leveraging the graph view, GNNs capture patterns across flows and hosts. For instance, a coordinated port scan or multi-hop attack will form a recognizable graph motif that the GNN can learn, whereas a conventional IDS examining isolated flows might miss it. In summary, GNNs provide a principled way to incorporate the rich relational context of network traffic into intrusion detection, potentially improving accuracy and resilience to evasion. The schematic flow diagram for the GNN-based IDS is depicted in Fig. 2. The diagram starts with data collection, gathering network traffic data, followed by a graph construction phase where nodes (e.g., devices, IPs) and edges (e.g., connections) are defined to model relational data. This is succeeded by feature extraction and preprocessing (e.g., embedding node/edge

Fig. 2 End-to-end workflow of a graph-based IDS, illustrating how raw network traffic between benign users and adversaries is transformed into a relational graph, enriched through node and edge feature encoding, and subsequently processed by a GNN model



attributes), tailored to the graph structure. The data then enters a GNN model phase, where GraphSAGE or GAT learn topological patterns to detect anomalies or classify attacks. The process concludes with detection results, triggering alerts and response actions (e.g., blocking or mitigation).

GNNs are most useful when the security signal depends on relationships that are not fully captured by flat features. Examples include repeated contact with a suspicious service, multi-hop lateral movement, coordinated low-rate scanning, graph-level botnet behavior, and temporal interaction patterns that evolve across sliding windows. In contrast, when the task is dominated by simple per-flow statistics or must run under strict data-plane memory constraints, non-graph or non-neural methods may be preferable. FlowLens shows that compact flow markers can support efficient ML-based flow classification directly on programmable switches, emphasizing that graph learning is not the only route to scalable traffic analysis (Barradas et al. 2021). The contribution of GNNs is therefore not that they replace all prior IDS methods, but that they become preferable when relational structure, message passing, and temporal graph context explain attack behavior better than isolated features.

1.3 Contributions of this survey

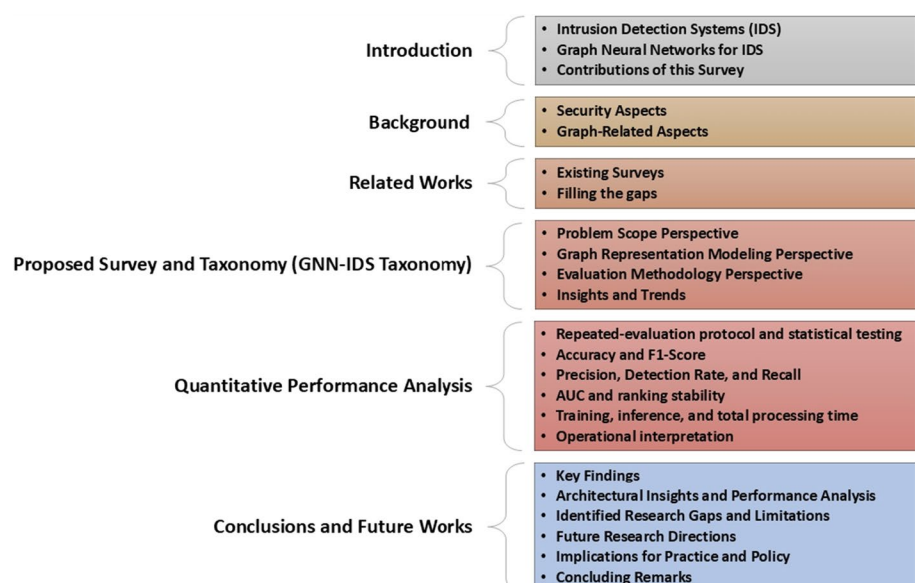
This survey makes the following contributions to the literature on GNN-based intrusion detection systems.

- First, the paper clarifies the conceptual boundary between GNN-based IDS and broader graph-based security analytics. Neural message-passing approaches are distinguished from non-neural graph methods such as random-walk analysis, rule-based graph traversal, knowledge graphs, provenance graphs, and causal graph analytics. This distinction helps clarify when GNNs are necessary, when they are merely optional, and when simpler graph or tabular methods may be more appropriate.
- Second, a deployment-aware taxonomy of GNN-based IDS research is proposed. The taxonomy connects attack setting, network environment, graph construction strategy, feature encoding, learning paradigm, task granularity, robustness consideration, dataset choice, evaluation protocol, and deployment constraints. This structure supports both literature organization and practical comparison across GNN-IDS studies.
- Third, the survey positions the present work against prior IDS, graph-based anomaly detection, and GNN-IDS review papers. Rather than treating all prior surveys as equivalent, the discussion focuses on the most directly related GNN-IDS reviews while using broader surveys to clarify the remaining research gap. This comparison highlights the need for a survey that combines graph-construction analysis, dataset-method mapping, baseline comparison, and operational evaluation.
- Fourth, the paper maps GNN methods to dataset characteristics and evaluation conditions. Flow-level datasets, IoT botnet traffic, provenance logs, temporal graphs, heterogeneous telemetry, and limited-label settings are shown to favor different graph constructions and GNN families. This mapping addresses the common problem

- of evaluating GNN architectures without considering whether the dataset structure actually supports the assumed graph inductive bias.
- Fifth, GNN-based IDS methods are compared with traditional machine-learning and deep-learning IDS families. Conventional baselines such as tree-ensemble, recurrent, convolutional-sequential, and anomaly-oriented IDS models are discussed not only as competitors, but also as necessary reference points for judging when graph message passing provides practical value.
 - Sixth, a leakage-controlled UNSW-NB15 quantitative case study is introduced. The evaluation uses temporal splitting, duplicate flow-label removal, endpoint-free topology construction, hard evaluation, normalized comparative visualization, and timing analysis. The evaluated models are treated as representative implementations of broader IDS families, allowing the analysis to compare both specific implementations and the methodological families they represent.
 - Seventh, deployment-oriented criteria are integrated into the performance discussion. Beyond Accuracy, F1-Score, Precision, Detection Rate, Recall, and AUC, the analysis considers training time, detection-time efficiency, total processing time, graph construction assumptions, and near-real-time feasibility. This helps shift the discussion from benchmark accuracy alone toward operational usefulness.
 - Finally, the survey identifies open research gaps in graph construction overhead, streaming and testbed-based evaluation, dataset fairness, adversarial robustness, explainability, cross-dataset generalization, and statistically rigorous benchmarking. These gaps define a practical research agenda for moving GNN-based IDS from promising benchmark models toward deployable cybersecurity systems.

The structure of the rest of the paper is as follows (Fig. 3). Section 2 introduces the background needed to interpret graph-based intrusion detection, including cyber-attack categories, graph representation learning, and graph modeling concepts. Section 3 reviews existing IDS, graph-based anomaly detection, and GNN-IDS surveys, with particular emphasis on the most directly related GNN-IDS reviews and the remaining research gap. Section 4 presents the proposed GNN-IDS taxonomy and discusses insights and trends across graph construction, feature encoding, learning paradigm, dataset usage, and deployment constraints. Section 5 reports a leakage-controlled quantitative evaluation on UNSW-NB15 using temporal splitting, duplicate-flow control, endpoint-free graph construction, normalized comparative visualization, and timing analysis. Section 6 concludes the paper by summarizing the main findings, identifying limitations, and outlining future research directions for robust and deployable GNN-based IDS.

Fig. 3 Structure of the paper



2 Background

This section provides the background needed to interpret GNN-based intrusion detection research. It first summarizes major cyber-attack categories and the security assumptions behind network-based intrusion detection. It then introduces graph representation learning and explains how network traffic can be modeled through nodes, edges, attributes, temporal dependencies, and heterogeneous entities. This background establishes the conceptual basis for the taxonomy and evaluation framework developed in the following sections.

2.1 Security aspects

2.1.1 Cyber-attacks classes

The cybersecurity landscape presents a sophisticated array of attack vectors. The following comprehensive taxonomy of cyber-attacks encompasses the primary threat categories encountered in network environments.

Advanced Persistent Threats (APTs) represent highly sophisticated, multi-stage attack campaigns orchestrated by well-resourced adversaries who establish persistent footholds within target networks. These attacks are characterized by their stealth, persistence, and strategic objectives, often involving lateral movement, privilege escalation, and data exfiltration over extended periods (Ren et al. 2023). APTs typically employ multiple attack vectors and exploit zero-day vulnerabilities to maintain undetected presence within compromised networks (Cuong et al. 2023; Ren and Geng 2025).

Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks aim to disrupt service availability by overwhelming target systems with excessive traffic or resource consumption requests. DDoS attacks leverage botnets to coordinate distributed traffic generation from multiple compromised hosts, making detection and mitigation more challenging than traditional DoS attacks (Saunders et al. 2024; Liu et al. 2021). Modern variants include application-layer attacks, reflection-amplification attacks, and carpet-bombing techniques that target multiple destinations simultaneously.

Botnet-driven campaigns utilize networks of compromised devices to execute coordinated malicious activities including spam distribution, credential stuffing, cryptocurrency mining, and DDoS attacks. The distributed nature of botnets and their peer-to-peer command-and-control architectures present significant detection challenges, as communication patterns may blend with legitimate network traffic (Alizadeh and Khansari 2023).

Malware-based attacks encompass various forms of malicious software including viruses, worms, trojans, ransomware, and advanced persistent malware. These attacks have evolved to include polymorphic and metamorphic variants that dynamically alter their code signatures to evade pattern-matching detection systems (Ruiz 2023b, a). Ransomware represents a particularly destructive subset that encrypts victim data and demands payment for restoration.

Web-based attacks target web applications and services through various vectors including Cross-Site Scripting (XSS), SQL injection, Cross-Site Request Forgery (CSRF), and session hijacking. These attacks exploit vulnerabilities in web application code and configuration to gain unauthorized access or manipulate application behavior (Liu et al. 2022).

Data exfiltration attacks involve the unauthorized extraction of sensitive information from compromised networks. These attacks often employ sophisticated techniques to avoid detection, including data fragmentation, encryption, steganography, and use of legitimate communication channels for covert data transmission (Sabir et al. 2022).

False Data Injection (FDI) attacks specifically target industrial control systems, smart grids, and IoT environments by manipulating sensor readings or control commands to deceive system operators or cause physical damage. These attacks are particularly concerning in critical infrastructure environments where operational integrity is paramount (Han et al. 2023; Das 2024).

Network scanning and reconnaissance attacks involve systematic probing of network infrastructure to identify live hosts, open services, and potential vulnerabilities. While individual probe attempts may appear benign, coordinated scanning activities can indicate preparation for more sophisticated attacks (Wang et al. 2023c).

Man-in-the-Middle (MitM) and spoofing attacks intercept or manipulate communications between legitimate parties through techniques such as ARP spoofing, DNS poisoning, and SSL/TLS interception. These attacks compromise communication integrity and confidentiality while potentially remaining undetected (Ha and Romero 2024).

Social engineering and phishing attacks, while primarily targeting human factors, often serve as initial attack vectors that lead to subsequent network intrusions through credential compromise or malware installation. These attacks frequently utilize emails, messaging platforms, and fraudulent websites to deceive victims (Akinyelu 2019).

Context-specific attacks target the unique vulnerabilities present in that context. For example, in IoT ecosystems, these attacks include weak authentication mechanisms, inadequate encryption, and insufficient update mechanisms. The resource constraints and diverse nature of IoT devices present additional challenges for traditional security mechanisms (Bibi et al. 2023; Ding et al. 2024).

Each attack category exhibits distinct behavioral signatures in terms of network traffic patterns and temporal characteristics. The effectiveness of GNN-based detection systems lies in their ability to model these complex relationships and dependencies that traditional flat-data approaches fail to capture.

2.1.2 Modern attack sophistication

Contemporary adversaries employ a diverse set of strategies to circumvent both signature-based and anomaly-based detection mechanisms. A first prominent method is evasion and obfuscation, whereby attackers deliberately fragment packets, encapsulate payloads within custom protocols, or exploit encryption in order to obstruct deep-packet inspection (You and Yim 2010). Equally important is the use of polymorphism and metamorphism, through which malware continuously mutates either its binary code or its behavioral patterns during propagation, thereby rendering static signatures largely ineffective (Alrzini and Pennington 2020). In addition, zero-day exploits remain a powerful means of bypassing defenses, as they take advantage of previously unknown vulnerabilities that have yet to be incorporated into existing signature databases (Deldar and Abadi 2024).

Advanced persistent threats (APTs) often materialize as multi-stage campaigns, typically progressing through reconnaissance, initial compromise, privilege escalation, lateral movement, and ultimately data exfiltration. Effective defense against such campaigns necessitates analytical models that are able to correlate and integrate events across these temporally distinct phases (Tankard 2011). Causal traffic-analysis systems such as Wedjat illustrate this point from a non-GNN perspective: rather than treating flows independently, they reason about causal relations among traffic events to expose sophisticated evasion behavior (Gao et al. 2025). Another increasingly prevalent tactic is the so-called living-off-the-land (LotL) approach, in which adversaries employ legitimate administrative tools -such as PowerShell or SS- to conceal malicious actions within otherwise normal system activity (Sudhakar 2020). Protocol tunneling and traffic shaping also complicate intrusion detection because malicious activity may be hidden behind packet-length or timing regularities rather than explicit payload signatures. Packet-length semantic analysis for tunneled flooding traffic is therefore relevant to IDS surveys even when it is not a GNN method, because it clarifies what can be captured by sequence- or packet-level representations before graph construction is introduced (Fu et al. 2024a). Taken together, these techniques underscore the inherent limitations of conventional intrusion detection systems. Signature-based solutions, while efficient against known threats, lack the capacity to recognize novel variants, whereas purely anomaly-based detectors are prone to generating excessive false positives in dynamic operational environments. Consequently, addressing these deficiencies requires the development of adaptive systems capable of learning complex relational patterns and evolving in parallel with adversarial behaviors.

2.2 Graph-related aspects

2.2.1 Graph representation learning (GRL)

Graph Representation Learning seeks to map graph elements -nodes, edges, or entire graphs- to low-dimensional vector spaces while preserving structural and attribute information (Zhang 2023):

- **Node Embeddings:** Encode each vertex (e.g., a host or IP address) into a feature vector such that similarly connected or behaving nodes lie close in embedding space.
- **Edge Embeddings:** Represent interactions (e.g., network flows) as vectors, facilitating tasks like anomalous link detection.
- **Graph (Subgraph) Embeddings:** Aggregate local or global graph structure into a single vector for high-level tasks such as campaign classification.

GRL methods range from shallow approaches (e.g., random-walk-based node2vec (Grover and Leskovec 2016)) to deep, end-to-end techniques (e.g., GNNs). In IDS, embeddings enable standard classifiers or clustering algorithms to operate on rich relational features rather than raw packet attributes alone.

2.2.2 Graph models for network data

Network traffic can be naturally modeled as graphs, where nodes represent entities (e.g. hosts, IPs, services) and edges represent communications or relationships (e.g. flows, connections) between them. Graph models may also be directed or undirected (reflecting whether flow direction is preserved), attributed (nodes/edges carry feature vectors), heterogeneous (multiple node/edge types), or have specialized structure (e.g. line or bipartite graphs). Each design choice affects what attack behaviors can be expressed and detected. In recent GNN-based IDS research, these varieties have been explored to capture multi-step, multi-modal, and context-rich intrusion patterns. In the following, we briefly summarize each graph modeling paradigm.

- **Static vs Dynamic Graphs** Static graphs encode a fixed snapshot of network traffic: all nodes and edges (e.g. IP hosts and their communications) are collected into one graph representing historical or aggregated data. In NIDS, static graphs are often used for offline analysis and vulnerability assessment, modeling the steady-state network structure (e.g. nodes = IPs, edges = all observed flows) (Xu et al. 2026). By contrast, dynamic (temporal) graphs explicitly model time-evolving traffic. One common approach is to split logs into time windows and build a sequence of graph "snapshots" ($G_1, G_2, \dots$) where each snapshot merges events in that interval (Xu et al. 2026). Dynamic graphs capture how edges and node features change over time, which is crucial for real-time IDS and detecting APTs that unfold gradually. In practice, GNN-IDS frameworks like Entente (Xu et al. 2026) use dynamic snapshots by default (enabling inductive learning on new data) but can easily be collapsed into a single static graph if needed. Static graphs are simpler and suit batch tasks (e.g. offline anomaly mining), whereas dynamic graphs suit online detection and temporal patterns (e.g. login-at-night anomalies)(Xu et al. 2026).
 - **Static graphs:** one fixed graph (e.g. combine all flows); captures stable relationships; good for retrospective IDS analysis. However, static models ignore evolving patterns and could yield coarse detection (all edges labeled at once).
 - **Dynamic (temporal) graphs:** time-aware graphs (e.g. sliding-window snapshots or continuous-time events); capture evolving topologies. GNNs (e.g. Temporal Graph Networks, RNN+GNN) process these to detect intrusions that develop over time (Van Langendonck et al. 2025).

- **Directed vs Undirected Graphs** Directed graphs have oriented edges (e.g. a flow from source to destination), whereas undirected graphs treat connections symmetrically. In network traffic, directed graphs naturally model one-way communications or attack flows. This retains who-initiated-whom in flows and enables edge-level classification (e.g. decide if that source→dest flow is malicious). In contrast, undirected graphs ignore direction and are often used when only connectivity matters (e.g. graph of co-occurring events or symmetric relationships). Directed modeling can reduce graph complexity and focus training (by cutting mutual-edge duplicates) (Li 2023). In IDS research, directed traffic graphs have been used in attention-based models (e.g. EDGMAT) to weight important source→target links (Li 2023). Undirected graphs appear in simpler settings (e.g. linking hosts with co-attack or similarity edges).
- **Heterogeneous Graphs** Heterogeneous graphs contain multiple nodes and/or edge types. In network IDS, this allows fusing different data modalities. For instance, Farrukh et al. (2025) build a heterogeneous graph with separate node types for flows and packet payloads, and use a heterogeneous GNN to jointly learn. Another example is integrating hosts, users, and services as distinct node types with relations between them. Heterogeneous graphs contain hosts, devices, flows, and alerts as different entities, with edges capturing different relations (e.g. "sent-flow", "logged-into"). By combining diverse information sources (flows, logs, packets, privileges, etc.) in one graph, a heterogeneous GNN can detect complex multi-stage attacks that cross these domains (Farrukh et al. 2025). Recent GNN-IDS works like XG-NID demonstrate that modeling both flow-level and packet-level features in one heterogeneous graph improves multi-class attack detection (Farrukh et al. 2025).
- **Emporal (Spatio-Temporal) Graphs** Temporal graphs (a type of dynamic graph) explicitly encode timestamps or evolving features on edges/nodes. These graphs capture both spatial (topological) and temporal dependencies. For example, PPT-GNN constructs a spatio-temporal traffic graph by incrementally adding flows as time progresses, enabling near-real-time predictions that adapt to temporal patterns (Van Langendonck et al. 2025). Such graphs may be built as continuous-time interaction networks (using Temporal Graph Networks, TGN) or as sequences of graphs with recurrent encoding. Temporal GNN-IDS models track how behaviors (e.g. a host's traffic volume) change, which assists flag time-based anomalies (e.g. scanning bursts or dormant-period intrusions).
- **Attributed Graphs** Attributed graphs are graphs where nodes and/or edges carry feature vectors. Almost all graph-based IDS use attributed graphs since network entities have associated data. In graph construction, nodes might represent devices/IPs with attributes (e.g. host type, role, recent load) and edges represent flows or interactions with attributes (e.g. byte count, protocol, frequency) (Xu et al. 2026). For example, Entente's graph construction uses node features like device type or privilege and edge features like flow volume and count (Xu et al. 2026). The GNN then learns over this attributed graph, combining topology with content. Attributed graphs enable the IDS to leverage richer signals: a GNN can use both structural patterns and known attributes (e.g. a server node's high privilege or a flow's unusual size) for anomaly scoring. Recent GNN-IDS models explicitly incorporate attributes such as EdgeGATConv concatenates node features with edge features in its attention computation (Li 2023), and NEGAT builds on GAT by injecting edge feature vectors into attention (Xu et al. 2024b).
- **Line Graphs** In a line graph representation, each edge of the original network graph is turned into a node in the line graph. Two line-graph nodes are connected if their corresponding edges in the original graph share a common endpoint. This "linearization" transfers flow attributes to the line-graph nodes. For example, Galli et al. (2025) note that by converting netflow edges to line-graph nodes, one can assign each flow's features (duration, ports, etc.) as node attributes and then apply node-classification techniques. In effect, edge-classification (is this flow malicious?) becomes node-classification on the line graph Galli et al. (2025). This approach has been adopted in IDS: GNN models like LineGraphSAGE operate on the flow-level line graph. The advantage is that it enables standard graph convolution on features that originate from flows (edges), and avoids ignoring edge attributes. The line graph approach is used in some models to boost training speed and accuracy on large

traffic graphs Galli et al. (2025). One trade-off is that the line graph can be much larger in size (each original edge becomes a node), although recent studies manage this in intrusion contexts.

- **Bipartite Graphs** Bipartite graphs split nodes into two disjoint sets, where edges connect two sets of nodes. In network IDS, common bipartitions include (User, Machine) or (Internal host, External host), (Source, Destination), or (Host, Service). For example, Entente's experiments use a user-machine bipartite graph for the LANL dataset: one node set is "users" and the other "machines", with edges if a user accessed a machine (Xu et al. 2026). Darknet and IoT flows are also sometimes modeled as source-destination bipartite graphs. Bipartite graphs explicitly separate roles to reveal which category of entity is involved in anomalies. They are suited for scenarios like login auditing or host-domain analysis: e.g. if one user suddenly connects to many servers (forming a star in a user-server bipartite graph), that pattern stands out. Bipartite modeling has appeared in IDS research for features engineering and synthetic data balancing (e.g. weighted SMOTE on bipartite structures (Xie et al. 2025)), and as a way to cluster or partition logs in federated GNIDS setups (Xu et al. 2026).
- **Edge-Aware Graphs** Edge-aware graphs explicitly incorporate edge attributes during learning. Edge-aware GNN models have been designed to use edge features directly in its message-passing layers (Xu et al. 2024b). NEGAT extends GAT by concatenating source-node, target-node, and edge embeddings to compute attention (Xu et al. 2024b). In practice, making GNNs edge-aware is important because the core IDS data lives on edges. Edge-aware graph models excel at flow-level tasks: e.g. edge classification (attack/not) or link prediction (will these hosts communicate in an attack?). By contrast, standard GNNs often ignore edge attributes or treat the graph as unweighted; adding edge-awareness allows the IDS to "see" things like unusual packet counts or malicious commands that are inherently edge attributes. Recent GNN-IDS designs like E-GraphSAGE/E-ResGAT explicitly augment GraphSAGE/GAT to process edge attributes, and have reported better intrusion detection performance by using edge features in aggregation (Xu et al. 2024b).

Graph-based intrusion detection should not be equated with GNN-based intrusion detection. Some graph-based methods rely on causal flow networks, flow-interaction graphs, random-walk analysis, provenance reasoning, or rule-based graph traversal without neural message passing. For example, real-time causal graph-analysis systems model between-flow causal relationships to detect sophisticated evasion attacks (Gao et al. 2025), while flow-interaction graph-analysis systems can identify unknown encrypted malicious traffic without relying on neural message passing (Fu et al. 2024b). These methods are graph-based, but they are not GNN-based. GNNs become preferable when the system must learn latent relational representations from high-dimensional node, edge, or temporal graph features rather than applying predefined graph rules or handcrafted causal structures.

As a matter of fact, it is important to distinguish graph-based IDS from GNN-based IDS. Graph-based IDS is the broader category: it includes rule-based graph traversal, graph queries, attack graphs, random-walk methods, knowledge graphs, provenance-path scoring, clustering, and centrality-based anomaly detection. These methods can be highly effective when detection logic is known in advance, when interpretability is the main requirement, or when the graph is small enough for explicit rules and path analysis. GNN-based IDS is narrower. It uses neural message passing to learn node, edge, or graph representations from labeled, weakly labeled, or self-supervised data. GNNs become preferable when attack behavior is relational, high-dimensional, partially hidden, or difficult to encode manually. They are less attractive when graph construction is expensive, labels are scarce without self-supervision, or the operational system requires strict deterministic rules.

Scope boundary

This survey therefore uses a layered scope. The core scope is GNN-based IDS, where the detector explicitly relies on neural message passing over a graph constructed from security telemetry. Broader graph-based security analytics are discussed when they clarify graph construction, provenance reasoning, causal flow analysis, or attack-path modeling, but they are not treated as GNN-IDS unless a learnable GNN layer is part of the detection pipeline. Dynamic and continual graph learning are included as enabling methodology for evolving network dependencies, distribution shift, and streaming updates. Adjacent spatio-temporal graph forecasting is treated

even more narrowly: it is cited only where it provides transferable ideas for temporal dependency modeling and is not counted as direct IDS evidence.

2.2.3 Adjacent spatio-temporal graph forecasting methods

Spatio-temporal graph forecasting is adjacent to, but distinct from, GNN-based intrusion detection. Its relevance to this survey lies in the way such models handle evolving graph dependencies, non-stationary signals, and temporally localized changes in networked systems. These issues also arise in streaming security monitoring, where traffic patterns, host interactions, and attack behaviors may shift over time. Representative traffic-forecasting studies show how spatially specific graph structures, multi-scale temporal dependencies, and adaptive recurrent graph states can be modeled when relationships among entities evolve over time (Yang and Wu 2024; Yang et al. 2025). Continual graph-learning studies further highlight catastrophic forgetting, distribution shift, streaming memory, and adaptive fine-tuning as core challenges for non-stationary graph environments (Tian et al. 2024; Wang et al. 2024). Practical pretraining work for network-security GNNs similarly motivates temporal representation learning for few-second graph windows and transfer to unseen networks (Van Langendonck et al. 2025). Accordingly, this survey uses these adjacent works to highlight transferable modeling principles for future adaptive and streaming GNN-IDS designs, while keeping the empirical and taxonomic focus on intrusion detection.

2.2.4 Graph neural networks (GNNs)

Intrusion Detection Systems (IDSs) based on graph neural networks (GNNs) typically follow a multi-stage pipeline: graph construction from network data, graph preprocessing and feature encoding, iterative message-passing GNN layers, and a final classification or anomaly-scoring output layer. In graph construction, raw network entities (such as hosts, IPs, packets or flows) are mapped to graph nodes, and relationships (e.g. network flows, protocol interactions) become edges. The edge is then endowed with attributes (flow features such as timestamp, port, packet size, and protocol), while node features may be initialized to simple embeddings (e.g. constant vectors) or derived from host metadata (Wang et al. 2023a). Some IDS designs use heterogeneous graphs (e.g. separate node types for hosts and services) or behavior-based graphs (e.g. BS-GAT treats each flow as a node and encodes "behavior rules" as edges) to capture specific domain structure. The goal is to capture relational patterns among entities such as hosts that communicate frequently or share similar behavior hence become tightly connected in the graph.

- *Nodes*: Often represent IPs, hosts, or logical entities in the network. Some works even treat *flows* as nodes (with edges encoding similarity or interaction).
- *Edges*: Usually represent communication (network flows) or inferred relationships. Edges may be directed or undirected; many GNN-IDS systems use undirected edges for simplicity. Edges carry rich attributes from the data flow (numeric features, one-hot or categorical features of protocols, flags, etc.).
- *Features*: Node features might encode host properties or be simple learnable vectors; edge features concatenate all flow attributes. Preprocessing often standardizes numerical features and one-hot encodes categorical.

Graph Preprocessing and Feature Encoding

Before feeding the data into the GNN, the raw network data are cleaned and encoded. Numeric features (e.g. byte counts, durations) are typically standardized (zero-mean, unit-variance) or log-transformed to handle skew (Gu et al. 2024). Categorical features (e.g. protocol types, flag categories) may be one-hot encoded, or in some cases dropped if they are unbounded (rare categories). (Gu et al. 2024) pre-process IDS datasets by dropping source-port (for anonymity), standardizing numeric fields, and one-hot encoding remaining categorical attributes. After preprocessing, the graph is constructed such that hosts (unique source/destination IPs) are nodes and flows are edges, with each edge's feature vector set to the concatenated attributes of that flow (Gu et al. 2024). In some pipelines, the graph may be dynamically updated in real time (sliding windows of flows) (Van Langendonck

et al. 2025). Sometimes graph augmentation or construction strategies are applied to improve robustness or capture domain patterns. For example, behavior-similarity graph building connects flows that share behavioral rules, ensuring each node has a uniform number of neighbors (Wang et al. 2023a). In practice, preprocessing may also involve splitting large graphs into manageable batches or sampling neighborhoods for scalability, as in GraphSAGE-style IDS pipelines (Lo et al. 2022).

GNN Layers: Message Passing, Aggregation, and Update

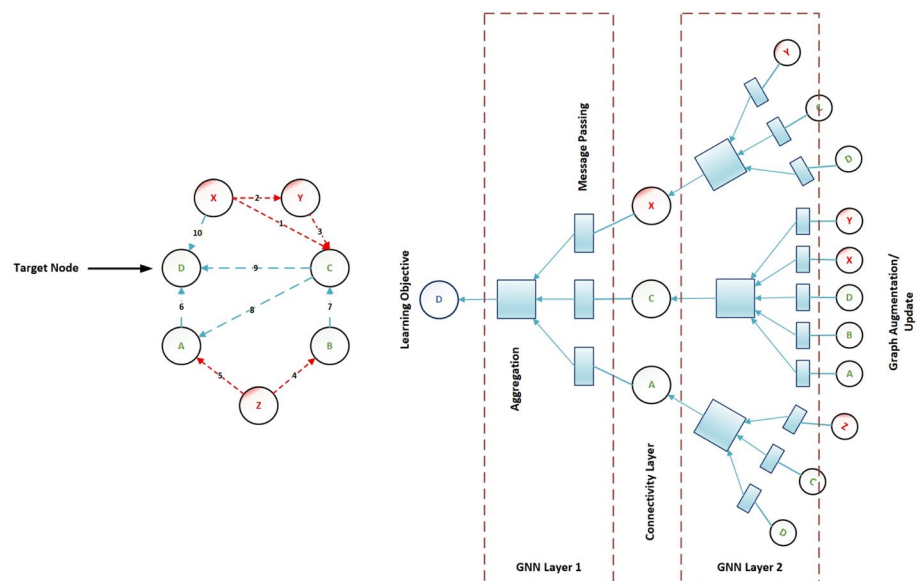
At the core of a GNN-based IDS is a message-passing network that iteratively updates node embeddings by exchanging information with neighbors.

Concretely, each GNN layer performs three steps: (1) **Message computation**: each node v sends a message to its neighbors based on its current embedding and the edge features. (2) **Aggregation**: each node collects (e.g. sums or averages) the incoming messages from all neighbors. (3) **Update/augmentation**: each node updates its own embedding by applying a neural update function (often an MLP) to the aggregated message and its previous embedding. Figure 4 shows the GNN main operations over the network. In practice, popular choices include:

- **Graph Convolutional Network (GCN) layers**: employ a simple weighted average of neighbor embeddings (often normalized by node degrees). They treat each neighbor equally (up to normalization) and apply a linear transform.
- **GraphSAGE layers**: considers a sample of fixed-size neighborhood and aggregate via mean or max pooling to supports large graphs processing via sampling (Lo et al. 2022).
- **Graph Attention Network (GAT) layers**: compute attention weights on each neighbor so that messages from highly relevant neighbors are more weighted (Ahanger et al. 2025; Wang et al. 2023a). In a GAT, the aggregation is a weighted sum where weights are learned via an attention mechanism (often multi-head attention).
- **Graph Isomorphism Network (GIN) layers**: use a sum aggregator with an injective update function, offering high representational power at the cost of more parameters.

Each of these layer types implements the same general message-passing paradigm (Gu et al. 2024). For example, GAT layers compute messages as linear projections of neighbors and then apply a learned attention to scale them before summing. Multiple such layers are stacked: deeper layers allow information to propagate across multi-hop paths (capturing longer-range traffic patterns), but very deep GNNs risk over-smoothing (where node embeddings become indistinguishable) or overfitting on noisy graph structures. Advanced variants may incorporate

Fig. 4 Generic two-layer GNN-based IDS architecture illustrating how the target node D iteratively refines its embedding through (i) message computation along directed edges with weighted connectivity, (ii) neighbor-aware aggregation over nodes A, B, C, X, Y, and Z in the connectivity layer, and (iii) update/augmentation in successive GNN layers that propagate security-relevant context across the graph



residual connections or jumping knowledge (concatenating or summing embeddings from different layers) to maintain expressivity in deeper models (Frenken et al. 2025).

Loss Functions and Output Layers

After several GNN layers, each node (or the entire graph) has a learned embedding. This is typically fed into a task-specific output layer. In a node-level IDS, a final linear layer maps each node's embedding to class scores (e.g. "benign" vs. "malicious" or specific attack categories) followed by a softmax. The training objective is usually the cross-entropy loss on the node labels (attack/no-attack or attack-type) (Frenken et al. 2025). If predicting attack type among C classes, one applies a softmax and uses multi-class cross-entropy. In imbalanced scenarios (e.g. far more normal flows than attacks), variants like weighted cross-entropy or focal loss are sometimes used (Frenken et al. 2025). Some GNN-IDS approaches use self-supervised or contrastive losses instead of (or in addition to) supervised labels. For instance, Xu et al. (2024b) propose a GNN with a contrastive loss that encourages embeddings of similar flow subgraphs to be close, enabling unsupervised identification of attack patterns. On the other hand, PPT-GNN is pre-trained to predict the existence of edges between nodes to improve its generalization ability (Van Langendonck et al. 2025). After pre-training, a small set of labeled examples can fine-tune the model via classification loss.

In summary, the output of the GNN is usually a vector of class logits, and the system is trained with a combination of classification losses (cross-entropy) and any auxiliary objectives (contrastive or reconstruction losses) tailored to the IDS context.

Architectural Variants and Their Impact

Different GNN architectures offer trade-offs in accuracy, scalability, and robustness:

- **Convolutional (GCN) vs. Attention (GAT):** GCN layers are simple and efficient (summing neighbor features) but cannot distinguish the importance of different neighbors. GAT layers weigh neighbors via attention, often improving detection of subtle patterns (e.g. pinpointing the most relevant flows) at the cost of more computation (Ahanger et al. 2025; Wang et al. 2023a). Empirically, attention can boost accuracy on complex intrusion graphs, while multi-head GATs are time consuming to train.
- **Sampling vs. Full Graph (GraphSAGE vs. GCN/GAT):** GraphSAGE's neighborhood sampling enables scalability to large networks by only aggregating a fixed number of neighbors per node (Lo et al. 2022). This accelerates training/inference on massive IDS data but may neglects some information by removing key neighbors in the sampling process. On the contrary, full-graph GCN/GAT read all neighbors (more accurate but slower on large graphs).
- **Depth and Message Aggregation:** Deeper GNNs can integrate multi-hop context, potentially catching distributed attacks that span many edges. However, deep stacking risks over-smoothing (node embeddings become similar) and overfitting to noise in the connectivity. Many IDS GNNs use only a few (2 to 4) layers. Advanced techniques like jumping-knowledge or residual links (seen in KD-GAT's use of jumping-knowledge) can mitigate depth issues (Frenken et al. 2025). The choice of aggregator (sum, mean, max, LSTM) also affects expressiveness: the more complex aggregators (like LSTM-attention) can capture better sequence-like interactions.
- **Heterogeneous and Multi-Modal GNNs:** Some works build heterogeneous graphs (e.g. distinct node types for flows vs. hosts, or combining packet- and flow-level graphs). XMF-GNN and similar designs employ attention to fuse different modalities to extract rich context (combining packet metadata and flow history), with boosting detection accuracy on complex data. The trade-off is between model complexity and training cost.
- **Temporal/Dynamic GNNs:** Traditional GNNs assume a static graph. For streaming IDS, dynamic GNNs incorporate time. PPT-GNN constructs graphs from short sliding windows of flows and optionally uses Temporal Graph Network (TGN) layers. By aggregating information across consecutive time-snapshots, these models better considered the evolving nature of attacks (Van Langendonck et al. 2025). Similarly, KD-GAT used a sliding window to build a temporal graph. These temporal extensions improve detection of time-dependent patterns but require additional design (e.g. RNN or temporal encodings) and often more data for training.

- **Model Compression/Distillation:** To reduce inference cost (important for real-time IDS), some studies train a large "teacher" GNN and then distill it into a smaller "student" GNN. The KD-GAT framework uses a full multi-layer GAT with jumping-knowledge as teacher and a very compact GAT as student (Frenken et al. 2025). The student learns via knowledge distillation (matching the teacher's soft logits) and can run with much lower latency while retaining accuracy. This is especially useful in embedded or high-throughput environments.
- **Feature Importance and Weighted Edges:** Many GNN-IDS incorporate edge weights or adaptive features. For example, BS-GAT assigns a learned weight to each edge based on behavioral similarity, so that important relationships have more influence (Wang et al. 2023a). Attention-based GNNs inherently reweight neighbors. These mechanisms can make the model more robust to noisy or spurious connections by down-weighting them.
- **Other GNN variants:** More specialized graph models have been explored as well. Graph Isomorphism Networks (GIN) are designed for maximal expressiveness in distinguishing graph structures. Graph Transformer models apply self-attention across all node pairs. Heterogeneous Graph Networks (HAN) can incorporate multiple node and edge types (e.g., differentiating routers, servers and IoT devices) by learning type-specific transformations. A Variational Autoencoder GNN (specifically, a Variational Graph Autoencoder, VGAE) detects anomalies by learning graph-aware latent representations of nodes that reconstruct normal patterns well. Meanwhile, nodes with large reconstruction errors or atypical latent variables are flagged as anomalies (Sardrolhefazi et al. 2024). Each architecture involves trade-offs: GCNs and GATs assume a mostly static graph, GraphSAGE and Transformers better handle growing graphs, and heterogeneous GNNs can model diverse node types. All these models share the core benefit of exploiting network connectivity in detection.

IDS-Specific Adaptations

GNN models for IDS often incorporate domain-specific adaptations beyond the standard pipeline:

- **Behavior-Based Graph Construction:** To exploit IDS semantics, some graph-building methods encode known protocols or rules. For instance, BS-GAT uses domain "behavior rules" as edges (so flows with the same rule share a link) and weights edges by behavioral similarity (Wang et al. 2023a). This yields a more uniform graph structure (each node has a similar neighbor count) and emphasizes meaningful correlations.
- **Graph Augmentation and Robustness Techniques:** Cybersecurity graphs can be noisy or adversarially manipulated. Recent work on adversarially robust GNN-IDS and structural perturbation highlights that edge manipulation, feature manipulation, and graph poisoning can affect detector reliability (Galli et al. 2025; Che et al. 2023; Xu et al. 2023). These add-ons help the IDS handle realistic adversarial conditions, but their effectiveness remains deployment-dependent.
- **Temporal Dynamics and Streaming:** Real-world traffic is dynamic. Models like PPT-GNN explicitly construct temporal snapshots (e.g. graphs of only a few seconds of traffic) to enable near-real-time detection (Van Langendonck et al. 2025). They also pre-train on large unlabeled flows (link prediction) to adapt model on new network environments with restricted labeled data. This addresses the challenge that standard GNN-IDS often rely on static graphs of hours of traffic, which is impractical for deployment.
- **Self-Supervised and Few-Shot Learning:** Labels for attacks can be limited or invalid (such as outdated ones). Accordingly, some GNN-IDS use self-supervision (contrastive or reconstruction losses) to learn from unlabeled traffic (Van Langendonck et al. 2025; Xu et al. 2024b). Then only a few labeled examples are needed to fine-tune. This improves adaptability to evolving threats and different network setups.
- **Handling Imbalanced and Evolving Classes:** Intrusions are rare events. Besides loss adjustments (focal or class weights), GNN-based IDS may use thresholding on output scores or one-class formulations (detect any anomaly). Some use graph autoencoders where the reconstruction error signals an outlier. These methods aim to improve recall on minority attack classes. (For example, KD-GAT notes that class imbalance significantly affected performance on one dataset (Frenken et al. 2025).)

- **Edge and Node Anomalies:** In host-based IDS, graphs might represent processes or system calls. GNN layers can be adapted to capture sequential patterns (e.g. using LSTMs as message functions). Other work (not cited here) uses attention over edge sequences.

In summary, a GNN-based IDS pipeline constructs a graph from traffic flows, preprocesses features, and runs one or more GNN layers (message passing + aggregation) to learn node embeddings. A final classifier (typically softmax) predicts intrusions. Variations in graph design (how nodes/edges are defined), in layer architecture (GCN/GAT/GraphSAGE depth, attention heads, etc.), and in training (loss functions, pre-training, distillation) all affect how well the system detects attacks, scales to large networks, and withstands noisy or adversarial conditions (Gu et al. 2024).

3 Related works

This section positions the present survey within the existing review literature on GNN-based intrusion detection and closely related graph-based security research. The discussion focuses primarily on the most directly related GNN-IDS surveys, while also considering broader IDS, graph-based anomaly detection, and explainable traffic-analysis reviews where they help clarify the remaining research gap. This positioning shows why a deployment-aware taxonomy, dataset-method mapping, traditional-baseline comparison, and leakage-controlled quantitative evaluation are still needed.

3.1 Existing surveys

Several surveys have recently been published to structure, evaluate, and forecast the landscape of GNN-based IDS research. Among them, two stand out as the most authoritative and frequently cited, each bringing a complementary perspective to the discussion.

The first is graph neural networks for intrusion detection: a survey by Bilot et al. (2023b). This paper offers a detailed review of GNN applications in both network- and host-based IDSs. The authors begin with a theoretical foundation, outlining diverse graph types -such as heterogeneous and temporal graphs- and modeling paradigms, including inductive versus transductive learning. Importantly, they synthesize classical random-walk-based methods with modern GNN architectures, presenting a comparative lens that enriches the understanding of their effectiveness. A distinguishing contribution of this work lies in its explicit focus on adversarial robustness and the unique challenges introduced by dynamic and obfuscated attack patterns. The strengths of this survey are reflected in its in-depth treatment of GNN algorithms, its classification of graph structures and learning paradigms, and its systematic comparison of spectral and spatial convolution approaches. Nonetheless, the technical complexity of the presentation may restrict its accessibility for practitioners, and its limited consideration of real-world deployment and scalability issues reduces its practical impact. Its novelty lies in being the first survey to formally integrate random-walk-based and GNN-based IDS models, as well as in explicitly incorporating adversarial robustness as a key dimension of comparison.

The second major work, a survey on graph neural networks for intrusion detection systems: methods, trends, and challenges by Zhong et al. (2024), introduces a comprehensive taxonomy of GNN-based intrusion detection frameworks. The paper classifies research along several axes, including graph construction, model architecture, training objectives, and evaluation strategies. Its approach effectively bridges theoretical foundations with practical challenges, highlighting issues such as dataset limitations, explainability, privacy, and deployment barriers. Unlike Bilot et al. (2023b), the emphasis here is less on algorithmic technicalities and more on constructing a broad organizational framework useful to both researchers and system designers. This work's strengths lie in its broad taxonomy, its alignment with real-world challenges, and its treatment of edge-, node-, and graph-level tasks. However, it provides less technical depth on specific GNN architectures and dedicates minimal attention

Table 1 Comparative overview of Bilot et al. (2023b) and Zhong et al. (2024) surveys

Survey	Focus	Strengths	Limitations	Novelty
Bilot et al. (2023b)	Methodological depth in GNNs for IDS	Provides rich coverage of GNN algorithms, a clear classification of graph structures, a detailed comparison of spectral and spatial learning, and explicit discussion of adversarial robustness	Highly technical in orientation, with comparatively limited attention to deployment constraints, operational scalability, and practical evaluation settings	Integrates random-walk-based graph analytics and GNN-based IDS within a unified discussion, with a notable emphasis on adversarial robustness
Zhong et al. (2024)	Taxonomy and research trends in GNN-based IDS	Offers a broad and structured taxonomy aligned with real-world concerns such as privacy, scalability, explainability, and node-, edge-, and graph-level detection tasks	Provides less architectural depth on individual GNN models and includes limited empirical treatment of adversarial evaluation	Unifies model design, data flow, and operational concerns in a practitioner-oriented synthesis of GNN-based IDS research

Table 2 Contextual positioning of the present survey against related IDS, graph-based anomaly detection, and GNN-IDS review studies

Paper	Year	Main focus	GNN/graph coverage	Limitation that motivates this survey
Khraisat et al. (2019)	2019	General IDS techniques, datasets, and challenges	Limited GNN-specific treatment	Does not provide a detailed graph-construction or message-passing taxonomy for GNN-based IDS
Nascita et al. (2025)	2024	Explainable AI for traffic classification and IDS	Covers explainable AI across traffic analysis and IDS	Explainability-centered rather than GNN-IDS taxonomy-centered; it does not systematically connect graph construction, GNN architecture, benchmark choice, and operational deployment
Ahmed et al. (2025)	2025	Consensus hybrid ensemble IDS with explainable AI	Non-GNN ensemble baseline perspective	Useful for SOTA positioning and table/metric presentation, but method-specific rather than a GNN-IDS survey
Ahmed et al. (2024)	2024	Hybrid bagging/boosting with SHAP-based feature selection for IDS	Non-GNN explainable tabular IDS	Supports the need to compare GNNs against strong tabular and ensemble baselines rather than assuming graph models are always superior
Ahmed and Sadiq (2026)	2026	Multi-hybrid deep-learning fusion and explainable AI for SDN IDS	Deep non-GNN IDS for SDN settings	Relevant to SOTA deployment positioning, but not focused on graph construction, graph leakage, or message-passing design
This survey	2026	GNN-based IDS taxonomy, benchmark interpretation, and deployment constraints	Covers GCN, GAT, GraphSAGE, edge-aware, temporal, heterogeneous, self-supervised, and hybrid GNNs	Adds a clearer GNN/non-GNN boundary, dataset-method mapping, timing-aware quantitative analysis, and reviewer-facing discussion of evaluation limitations

to adversarial evaluations. Its novelty stems from being the first to formalize a taxonomy that simultaneously addresses model design, data flow, and operational concerns, while also synthesizing research trends and future directions in a manner accessible to practitioners.

Taken together, these two surveys provide complementary perspectives: Bilot et al. (2023b) emphasize methodological depth and adversarial robustness, while Zhong et al. (2024) highlight taxonomy building and operational alignment. Yet, both leave gaps that warrant further exploration -whether in terms of scalability, adversarial resistance, or the integration of broader methodological concerns into practical deployment scenarios (see Table 1). Addressing these shortcomings motivates the need for a unified survey that not only synthesizes methodological advances but also provides a comprehensive taxonomy capable of bridging theoretical and operational dimensions.

Although this survey primarily positions itself against the two most directly related GNN-IDS surveys, a broader contextual comparison is useful to clarify the remaining research gap. Existing reviews either cover intrusion detection in general, focus on graph-based anomaly detection more broadly, or emphasize adjacent concerns such as explainability and traffic classification. Table 2 therefore does not treat all prior surveys as equally close competitors; rather, it situates the present work within the wider review landscape and highlights the specific gap addressed here: a deployment-aware GNN-IDS taxonomy combined with graph-construction analysis, dataset-method mapping, traditional-baseline comparison, and leakage-controlled quantitative evaluation.

As shown in Table 2, the closest points of comparison are the GNN-IDS-focused surveys, which provide valuable summaries of architectures and application domains. However, they provide less systematic treatment of deployment constraints, graph-construction leakage, dataset-method alignment, and direct comparison with strong non-GNN IDS baselines. The present survey therefore does not replace these works, but extends them by connecting GNN design choices to dataset properties, evaluation fairness, and practical deployment requirements.

The broader traffic-analysis and recent SOTA IDS literature also help define the boundary of this survey. Work on automated traffic analysis emphasizes that robust traffic classifiers must account for deployment context, data representation, and adversarial adaptation rather than only benchmark accuracy (Holland et al. 2021). Recent ensemble, SHAP-based, and SDN-oriented deep-learning IDS studies further show that carefully engineered non-GNN models can remain highly competitive when discriminative information is already well represented in tabular, sequential, or fused feature spaces (Ahmed et al. 2025, 2024; Ahmed and Sadiq 2026). Similarly, packet-length semantic modeling and cross-modality feature fusion show that non-GNN deep models can be effective when the discriminative signal is mainly sequential, spectral, or multi-modal rather than explicitly relational (Fu et al. 2024a, 2025). These studies motivate the inclusion of conventional and non-graph baselines in the quantitative analysis: they provide a necessary reference point for deciding when graph message passing is actually justified.

3.2 Filling the gaps

While the surveys by Bilot et al. (2023b) and Zhong et al. (2024) provide valuable insights into GNN-based IDS research, they also leave notable gaps that limit a comprehensive understanding of the field. Our survey is designed to complement these works by offering a more structured and systematic analysis of GNNs for IDSs, framed along several key dimensions.

Problem Domain Specialization: Unlike earlier surveys that treat intrusion detection broadly, this work organizes attack types within specific network contexts. The analysis distinguishes attacks that are directly benchmarked in the cited GNN-IDS literature from attacks that are mainly used as motivation or deployment scenarios. This distinction is important because a survey taxonomy can catalog APTs, botnets, DDoS, web attacks, false-data injection, and reconnaissance, while a single empirical benchmark may validate only a subset of these categories.

Comprehensive Graph Structure Analysis: While existing surveys categorize graphs as static or dynamic, this work provides systematic discussion of graph structure trade-offs and their performance implications. We analyze how node-centric, edge-centric, temporal, and hybrid feature encodings affect what information is available to the detector. The analysis also separates endpoint-based graphs from endpoint-free flow-similarity graphs, because these choices imply different leakage risks, evasion assumptions, and runtime costs.

Deployment-Aware Comparison: A practical IDS is constrained by packet capture, feature extraction, state maintenance, memory, latency, and where the detection logic runs in the network. Systems such as FlowLens and NetBeacon demonstrate that efficient flow classification and programmable data-plane design can be as important as the downstream classifier itself (Barradas et al. 2021; Zhou et al. 2023). This survey therefore treats inference time, training time, graph construction, and deployment feasibility as first-order dimensions rather than as secondary implementation details.

Multi-Granularity Detection Framework: This survey introduces a unified multi-granularity framework that includes graph-, subgraph-, node-, edge-, and flow-level analysis. Different granularities are compared conceptually rather than treated as directly interchangeable, because a model designed for supervised flow classification does not share the same threat model as a self-supervised graph anomaly detector or a provenance-based campaign detector.

Learning Paradigm Systematic Comparison: This work provides comparative analysis across supervised, semi-supervised, self-supervised, and unsupervised paradigms. The discussion emphasizes how labeling requirements shape deployment assumptions: supervised flow classifiers are appropriate for known attack families with

labeled data, whereas anomaly-oriented or self-supervised graph models are more relevant when zero-day behavior, label scarcity, or delayed labels dominate the setting.

Enhanced Evaluation Methodology: Beyond traditional IDS metrics, this survey emphasizes graph-specific reporting factors, including node and edge definitions, temporal windows, graph construction time, leakage-control mechanisms, edge-list growth, and update frequency. These factors do not replace Accuracy, F1-Score, Precision, Recall, Detection Rate, and AUC; rather, they make it possible to interpret those metrics under fairer graph-based and non-graph comparison conditions.

Systematic Dataset-Attack Analysis: This work maps dataset families to attack types and graph-construction assumptions, explaining which datasets are suitable for evaluating specific categories and which attack classes remain weakly validated. The empirical study is intentionally narrower: it provides a controlled UNSW-NB15 case study rather than a universal validation of all attack categories catalogued in the taxonomy.

Empirical Case Study: Unlike purely theoretical surveys, this work includes a leakage-controlled performance comparison of representative IDS model families on UNSW-NB15. The case study is used to illustrate operational trade-offs, such as detection rate versus false positives and graph construction overhead versus classifier runtime, rather than to claim that one benchmark establishes universal superiority across all network environments.

In summary, this work provides the specialized, systematic, and empirically-validated framework necessary for advancing GNN-based intrusion detection from research to practical deployment. These innovations collectively address the gap between theoretical understanding and operational implementation in cybersecurity applications.

Key Methodological Distinctions

The innovations presented in this survey are not merely incremental improvements but represent fundamental shifts in how GNN-based IDS research is approached:

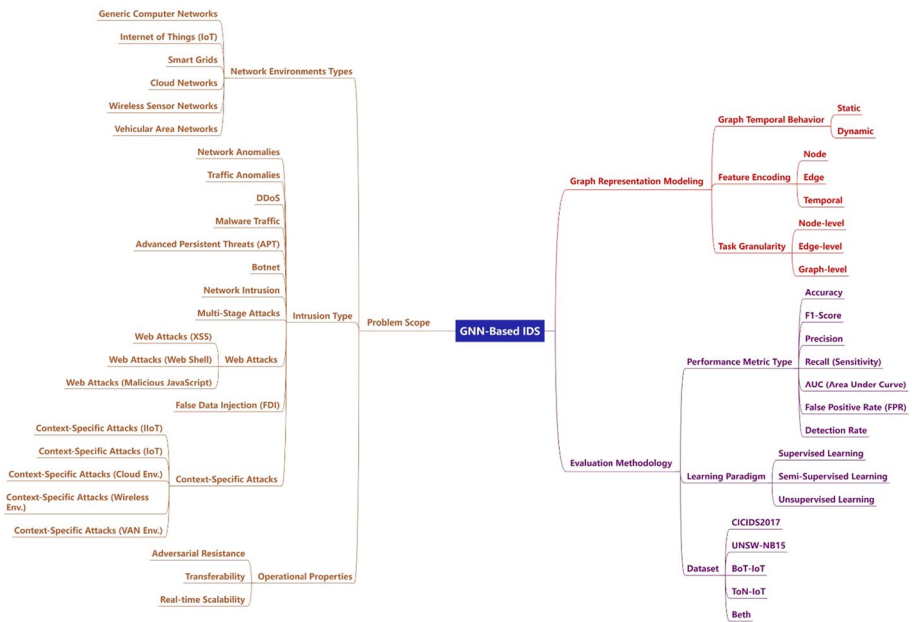
1. **From Generic to Domain-Specific:** Moving beyond general frameworks to provide specialized analysis for different network environments and attack types.
2. **From Categorical to Comparative:** Replacing simple categorizations with systematic comparisons and performance trade-off analyses.
3. **From Isolated to Unified:** Integrating different analytical dimensions (granularity, paradigms, structures) into coherent frameworks.
4. **From Theoretical to Empirical:** Supplementing theoretical analysis with practical implementation and validation.
5. **From Descriptive to Prescriptive:** Providing actionable recommendations rather than purely descriptive overviews.

These methodological innovations position this survey as a bridge between theoretical research and practical implementation, addressing the current disconnect between academic advances and operational cybersecurity needs.

4 Proposed survey and taxonomy (GNN-IDS taxonomy)

This section presents the proposed GNN-IDS taxonomy (see Fig. 5). The taxonomy organizes prior work across problem scope, graph construction, feature encoding, learning paradigm, task granularity, robustness, evaluation methodology, and deployment constraints. It is intended both as an analytical framework for comparing existing studies and as a guide for identifying underexplored research directions in graph-based intrusion detection.

Fig. 5 Proposed taxonomy for GNN-based IDS (GNN-IDS Taxonomy), organizing the literature around problem scope (network environments and intrusion types), graph representation modeling (graph temporal behavior, feature encoding at node/edge/temporal levels, and task granularity), and evaluation methodology (learning paradigms, datasets, and performance metrics)



4.1 Problem scope perspective

The problem scope of this survey includes the types of intrusions that the NIDS are designed to detect and the environments in which they operate.

4.1.1 Network environments types

IDSs are designed for a range of environments, including Generic computer networks, IoT networks, wireless sensor networks (WSNs), cloud environments, Industrial Internet of Things (IIoT), smart grids and in-vehicle networks.

4.1.2 Intrusion types

Various intrusion types are considered, such as Advanced Persistent Threats (APTs), botnets, malware, Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks, web attacks like Cross-Site Scripting (XSS), and false data injection attacks. Some papers focus on specific attacks like ransomware or those targeting the Vehicular Area Networks (VAN) in vehicles.

4.1.3 Operational properties

This category encompasses the system's ability to maintain effective operation and performance under various challenging conditions, including hostile environments, changing operational contexts, and demanding performance requirements.

1. **Adversarial Resistance** Adversarial resistance in GNN-based intrusion detection systems refers to the model's capacity to maintain consistent detection performance when subjected to carefully crafted perturbations designed to evade detection or cause misclassification (Che et al. 2023). According to recent literature, adversarial resistance can be mathematically expressed as the stability of the GNN function f under bounded perturbations. For a GNN-based IDS with function $f(G)$ operating on graph G , adversarial resistance is achieved when:

$$\|f(G) - f(G')\| \leq \varepsilon$$

where G' represents the adversarially perturbed graph with perturbations bounded by some constraint set Δ , and ε represents the acceptable deviation threshold. In GNN-based IDS, adversarial resistance manifests as the system's ability to correctly identify malicious network activities even when attackers attempt to manipulate network traffic patterns, connection structures, or feature representations to evade detection. This includes resistance to both evasion attacks (during inference) and poisoning attacks (during training) (Xu et al. 2023; Ennaji et al. 2025).

2. **Transferability** Transferability in GNN-based intrusion detection systems refers to the ability of a trained model to maintain effective detection performance when deployed across different network environments, topologies, or operational contexts without requiring complete retraining (Ruiz 2023b, a). Following the graphon-based framework established in the literature, transferability can be formally defined as: For two graphs G_1 and G_2 that approximate the same underlying phenomenon (represented by a common graphon $\mathcal{W}$), a GNN-based IDS ϕ is transferable if Ruiz (2023b), Ruiz (2023a):

$$\|\phi(G_1) - \phi(G_2)\| = O(n_1^{-0.5} + n_2^{-0.5})$$

where n_1 and n_2 are the sizes of the respective graphs regarding their number of nodes, ensuring that the error decreases as graph sizes increase (Ruiz 2023a). In cybersecurity applications, transferability enables a GNN-based IDS trained on one network topology or organization to effectively detect intrusions in different network configurations, scales, or even across different domains (e.g., from enterprise networks to IoT environments) (Tian et al. 2025; Buterez et al. 2024).

3. **Real-time Scalability** Real-time scalability can be characterized by two key metrics:

- **Latency constraint:** $L(n) \leq L_{\max}$ where $L(n)$ is the processing latency for a graph of size n and $L_{\max}$ is the maximum acceptable latency.
- **Throughput requirement:** $T(n) \geq T_{\min}$ where $T(n)$ is the system throughput for graph size n and $T_{\min}$ is the minimum required throughput (Liu et al. 2015).

For GNN-based IDS, real-time scalability ensures that the system can handle increasing network traffic volumes, grow numbers of network nodes, or expand connection patterns while maintaining detection capabilities within operational time constraints. This includes achieving sub-second response times for threat detection even in large-scale networks (Buterez et al. 2024).

Next, Table 3 contrasts the prominent articles examined under these criteria.

Table 3 should be read as a scope map rather than as proof that every listed attack family has been evaluated with equal depth. In particular, the literature often motivates multi-stage attacks, APTs, encrypted malware traffic, or adversarially adapted behavior, while the actual benchmark may still be a binary benign/attack classification task on a public dataset. Table 4 therefore summarizes the validation boundary used in this survey. The purpose is to distinguish attacks that are genuinely benchmarked at a specific granularity from attacks that are mainly discussed as deployment motivation.

4.2 Graph representation modeling perspective

This category emphasizes how the graph is constructed and used by the model, including whether the graph is static or dynamic, how features are encoded on nodes/edges (or in time), and at what granularity the model makes predictions.

Table 3 Problem-scope mapping of GNN-based IDS studies across network environments, summarizing for each work the targeted intrusion type and whether key operational properties, including adversarial resistance, transferability, and real-time scalability, are explicitly addressed or left unexplored

Network Env. type	References	Intrusion type	Operational properties
Generic Computer Net	Bilot et al. (2023a)	Network anomalies	None
Generic Computer Net	Caville et al. (2022)	Network anomalies	Adversarial resistance
Generic Computer Net	Venturi et al. (2023)	Network anomalies	Adversarial resistance
Generic Computer Net	Chang and Branco (2024)	Network anomalies	Adversarial resistance, transferability
Generic Computer Net	Le and Park (2024)	Network anomalies	Transferability
Generic Computer Net	Sun et al. (2024)	Network anomalies	Transferability, real-time scalability
Generic Computer Net	Du et al. (2022)	Traffic anomalies	Transferability
Generic Computer Net	Liu et al. (2021)	DDoS	Real-time scalability
Generic Computer Net	Abinesh et al. (2024)	Malware traffic	Transferability
Generic Computer Net	Chen et al. (2024)	Malware traffic	Transferability, real-time scalability
Generic Computer Net	Zhang et al. (2023)	Malware traffic	Transferability
Generic Computer Net	Mora-Gimeno et al. (2021)	Malware traffic	Transferability
Generic Computer Net	Ren and Geng (2025)	Advanced persistent threats (APT)	Real-time scalability
Generic Computer Net	Cuong et al. (2023)	Advanced persistent threats (APT)	Transferability, real-time scalability
Generic Computer Net	Alizadeh and Khansari (2023)	Botnet	Adversarial resistance
Generic Computer Net	Deng and Huang (2025)	Network intrusion	None
Generic Computer Net	Gorricho-Segura et al. (2023)	Network intrusion	Transferability
Generic Computer Net	King et al. (2023); Farrukh et al. (2025)	Network intrusion	Various
Generic Computer Net	Chen et al. (2023)	Multi-stage attacks	Transferability
Generic Computer Net	Pan et al. (2024)	Web attacks (XSS)	Adversarial resistance, transferability, real-time scalability
Generic Computer Net	Feng et al. (2024)	Web attacks (Web Shell)	Adversarial resistance, transferability, real-time scalability
Generic Computer Net	Liu et al. (2022)	Web attacks (XSS)	Transferability, real-time scalability
Generic Computer Net	Fang et al. (2022)	Web attacks (malicious JavaScript)	Adversarial resistance, transferability, real-time scalability
Generic Computer Net	Bao et al. (2023)	Web attacks	Transferability, real-time scalability
Generic Computer Net	Xu et al. (2023)	Web attacks	Adversarial resistance, transferability, real-time scalability
Generic Computer Net	Li et al. (2023a); Liu et al. (2018)]	Network anomalies and intrusions	Various
Internet of Things (IoT)	Singh et al. (2024)	Network anomalies	None
Internet of Things (IoT)	Bhumika and Das (2022)	Network anomalies	None
Internet of Things (IoT)	Han et al. (2024)	Network anomalies	Transferability, real-time scalability
Internet of Things (IoT)	Das (2024)]	False data injection (FDI)	None
Internet of Things (IoT)	Guo and Li (2024)	Advanced persistent threats (APT)	Adversarial resistance, transferability
Internet of Things (IoT)	Ren et al. (2023)]	Advanced persistent threats (APT)	Real-time scalability
Internet of Things (IoT)	Maksimowski (2023)	Botnet	None
Internet of Things (IoT)	Shareef et al. (2024); Lo et al. (2023)	Botnet	Various
Internet of Things (IoT)	Bibi et al. (2023)]	Network intrusion	Transferability, real-time scalability
Internet of Things (IoT)	Ding et al. (2024); Boukari et al. (2024)	Network intrusion	Various
Internet of Things (IoT)	Bora and Chauhan (2023)	Context-specific attacks (IIoT)	None
Internet of Things (IoT)	Manikandan et al. (2024); Ullah et al. (2023)	Context-specific attacks (IIoT)	Various
Internet of Things (IoT)	Okui et al. (2023)	Network anomalies	Transferability
Internet of Things (IoT)	Long et al. (2023); Latif et al. (2023)	Network anomalies	Transferability
Smart Grids	Lv et al. (2023)	Network anomalies	Transferability, real-time scalability
Smart Grids	Liu et al. (2023)	Traffic anomalies	Real-time scalability
Smart Grids	Tang et al. (2022)	False data injection (FDI)	Transferability

Table 3 (continued)

Network Env. type	References	Intrusion type	Operational properties
Smart Grids	Lakshmanan et al. (2024); Wei et al. (2023); Chen et al. (2023); Boyaci et al. (2022)	False data injection (FDI)	Various
Smart Grids	Takiddin et al. (2023b)	Malware traffic	None
Cloud Networks	Jianping et al. (2024); Lakha et al. (2022)	Network anomalies	Transferability, real-time scalability
Cloud Networks	Saunders et al. (2024)	DDoS	Transferability, real-time scalability
Cloud Networks	Barsellotti et al. (2023)	DDoS	Real-time scalability
Cloud Networks	Li et al. (2022)	DDoS	Transferability, real-time scalability
Cloud Networks	Tang et al. (2024)	Malware traffic	None
Cloud Networks	Abdullayeve and Suleymanzade (2024)	Context-specific attacks (Cloud Env.)	Transferability
Wireless Sensor Net	Kumar and Khari (2025); Gharavian et al. (2023)	Context-specific attacks (Wireless Env.)	Transferability
Wireless Sensor Net	Biswas et al. (2023); Ha and Romero (2024)	Context-specific attacks (Wireless Env.)	Transferability, real-time scalability
Vehicular Area Networks	Zambare and Liu (2024)	Context-specific attacks (VAN Env.)	Transferability, real-time scalability
Vehicular Area Networks	Song et al. (2024)	Context-specific attacks (VAN Env.)	Adversarial resistance, real-time scalability
Vehicular Area Networks	Park et al. (2023)	Context-specific attacks (VAN Env.)	Transferability
Vehicular Area Networks	El-Dawy (2024)	Context-specific attacks (VAN Env.)	Transferability, real-time scalability

Table 4 Attack-category validation boundary across the survey taxonomy and the UNSW-NB15 case study

Attack category	How it appears in the surveyed literature	Typical evaluated granularity	Validation boundary in this survey
Generic network intrusion and anomaly traffic	Frequently benchmarked on CICIDS2017, UNSW-NB15, BoT-IoT, ToN-IoT, and related flow datasets	Flow-, node-, edge-, or graph-level binary/multi-class detection	Directly represented in the taxonomy and closest to the empirical UNSW-NB15 binary case study
Botnet, DDoS, reconnaissance, DoS, exploits, and malware traffic	Commonly evaluated in public IDS datasets, although attack labels and class imbalance vary substantially	Mostly flow-level or host/flow graph classification	Evaluated in cited works when datasets expose these labels; the present case study reports aggregate binary detection rather than per-category attack breakdown
Web attacks, false-data injection, vehicular, smart-grid, and industrial attacks	Usually tied to domain-specific telemetry and specialized graph construction	Node-, edge-, or event-level detection depending on the domain	Included in the taxonomy as domain-specific GNN-IDS evidence, but not validated by the UNSW-NB15 case study
APT, multi-stage campaigns, provenance attacks, and lateral movement	Often motivated by graph reasoning, provenance paths, temporal dependencies, and cross-event correlation	Subgraph-, path-, campaign-, or event-sequence level	Treated as important deployment scenarios, but near-perfect binary benchmark scores should not be extrapolated to these settings without campaign-level evaluation
Zero-day, adaptive, encrypted, and adversarially modified traffic	Discussed in anomaly, self-supervised, robustness, and encrypted-traffic studies	Often anomaly score, temporal drift, or behavior-level detection	Considered an open validation frontier; static public benchmarks cannot by themselves establish robustness to distribution shift or adaptive evasion

4.2.1 Graph temporal behavior

Static graphs have a fixed set of nodes and edges representing the network's steady-state structure (e.g. an attack graph or host network topology). They capture stable relationships but do not change over time. In contrast, dynamic graphs evolve as new edges or nodes appear/disappear or as attributes change. Dynamic graphs model temporal evolution (such as network flow activity or time-stamped interactions) and are essential for real-time IDS tasks. For example, Zhong et al. (2024) note that static graphs "represent the steady-state structure of a network or host" and are suited to offline or historical analysis, whereas dynamic graphs "model the temporal aspects of network traffic and system behavior" and capture sophisticated attacks that unfold over time. In other words, static versus dynamic graph behavior refers to whether the graph structure (and attributes) is fixed or time-varying. GNN-IDS studies often specify which they use: e.g. Sun et al. (2024) build a static "attack graph" for known vulnerabilities, and overlay dynamic real-time traffic on it.

4.2.2 Feature encoding

Node features are attribute vectors attached to each node (e.g. host properties, IP addresses, payload characteristics). Edge features are attributes associated with edges (e.g. flow statistics, connection frequencies, or protocol types). Many GNN-IDS methods explicitly encode both such as, Altaf et al. (2023) considered node features as "application-level information specific to packet content" and edge features as "network-level information" (network flows). In practice, node features might include host logs or payload byte histograms, while edge features might include traffic volume or timing between two hosts. Temporal features capture time-related aspects of the graph (timestamps, evolving values, or sequential patterns). Temporal encoding might involve timestamping edges (for continuous-time dynamic graphs) or adding time windows of activity as features. Dynamic GNN surveys explain that in dynamic graphs "nodes, edges, and properties, demonstrate continuous change over time" (Zheng and Wei 2024). Thus, temporal features are simply attributes that vary with time (e.g. the time since last contact, packet inter-arrival times, or sliding-window statistics). In summary, feature encoding in GNN-IDS refers to *what information is encoded on the graph*: node-level attributes, edge-level attributes, and any time-related attributes to capture dynamics (Zhong et al. 2024).

4.2.3 Task granularity (node-level, edge-level, graph-level)

Graph-based IDS models can make predictions at different scales. Node-level tasks classify or regress on each individual node (e.g. labeling compromised hosts) (Zhong et al. 2024). Edge-level tasks classify or score each edge (e.g. labeling a traffic flow as malicious or benign). Graph-level tasks classify an entire graph (e.g. determining if the overall network state or session is malicious). For example, Zhong et al. (2024) define node-level tasks as "predictions or classifications for individual nodes", edge-level tasks as predictions related to the edges or connections (often computed from connected node representations), and graph-level tasks as making predictions for the entire graph via a global readout. In other words, task granularity indicates whether the IDS model outputs a label for each node, for each edge, or for each whole graph. GNN-IDS papers explicitly state this granularity (e.g. design an edge-attention model for edge-level flow classification).

In Table 5, a comparative table delineates the key papers analyzed based on these features.

4.3 Evaluation methodology perspective

The evaluation methodology of graph-based intrusion detection methods requires a clear, standardized presentation of how each study was evaluated. In this section, we group together three complementary aspects of experimental design-performance metric type, learning paradigm, and dataset to emphasize that these items describe

Table 5 Graph representation modeling characteristics of GNN-based IDS studies, categorizing each work by graph temporal behavior (static versus dynamic), the primary feature encoding strategy (node, edge, temporal, or hybrid), and task granularity (node-, edge-, or graph-level prediction), thereby clarifying how design choices in graph construction and feature placement shape the level at which intrusions are detected and the types of temporal patterns each model can capture

Graph temporal behavior	References	Feature encoding	Task granularity
Static	Bilot et al. (2023a)	Node	Node-Level
Static	Caville et al. (2022)	Edge	Edge-Level
Static	Chang and Branco (2024)	Node	Node-Level
Static	Liu et al. (2021)	Node	Node-Level
Static	Takiddin et al. (2023b)	Node	Node-Level
Static	Abinesh et al. (2024)	Edge	Edge-Level
Static	Zhang et al. (2023)	Edge	Edge-Level
Static	Alizadeh and Khansari (2023)	Node	Node-Level
Static	Gorricho-Segura et al. (2023)	Edge	Edge-Level
Static	Han et al. (2024)	Temporal	Graph-Level
Static	Mirlashari and Rizvi (2024)	Temporal	Graph-Level
Static	Kumar and Khari (2025)	Node	Edge-Level
Static	Gharavian et al. (2023)	Edge	Edge-Level
Static	Feng et al. (2024)	Temporal	Node-Level
Static	Xu et al. (2024b)	Temporal	Edge-Level
Static	Zhou et al. (2020)	Node	Graph-Level
Static	Bilot et al. (2024)	Edge	Graph-Level
Static	Zoubir and Missaoui (2025)	Node	Node-Level
Static	Wang et al. (2021)	Node	Node-Level
Static	Kisanga (2023)	Node	Node-Level
Static	Zola et al. (2022)	Node	Node-Level
Static	Xu et al. (2024a)	Edge	Edge-Level
Static	Hassan et al. (2023)	Edge	Edge-Level
Dynamic	Bhumika and Das (2022)	Node	Node-Level
Dynamic	Venturi et al. (2023)	Edge	Edge-Level
Dynamic	Han et al. (2024)	Temporal	Graph-Level
Dynamic	Le and Park (2024)	Temporal	Graph-Level
Dynamic	Sun et al. (2024)	Node	Node-Level
Dynamic	Du et al. (2022)	Edge	Edge-Level
Dynamic	Li et al. (2022)	Node	Node-Level
Dynamic	Guo and Li (2024)	Edge	Edge-Level
Dynamic	Lo et al. (2023)	Temporal	Graph-Level
Dynamic	Ji and Huang (2022)	Temporal	Graph-Level
Dynamic	Wang et al. (2023c)	Node	Graph-Level
Dynamic	Li (2023)	Edge	Graph-Level
Dynamic	Zhang et al. (2022)	Edge	Edge-Level
Dynamic	Pujol-Perich et al. (2022)	Node	Graph-Level
Dynamic	Ding et al. (2024)	Edge	Edge-Level
Dynamic	Lo et al. (2022)	Edge	Edge-Level
Dynamic	Manjula et al. (2023)	Edge	Edge-Level
Dynamic	Messai and Seba (2023)	Node	Graph-Level
Dynamic	Boukari et al. (2024)	Temporal	Graph-Level
Dynamic	Zambare and Liu (2024)	Node	Node-Level
Dynamic	Song et al. (2024)	Node	Node-Level
Dynamic	Park et al. (2023)	Edge	Edge-Level
Dynamic	El-Dawy (2024)	Edge	Graph-Level
Dynamic	Wu et al. (2024)	Temporal	Graph-Level
Dynamic	Peng and Zhang (2024)	Temporal	Graph-Level
Dynamic	Xu et al. (2023)	Edge	Edge-Level
Dynamic	Marfo et al. (2024)	Temporal	Graph-Level
Dynamic	Casas et al. (2023)	Node, Edge, Temporal	Edge-level, Node-Level, Graph-Level

Table 5 (continued)

Graph temporal behavior	References	Feature encoding	Task granularity
Dynamic	Roy and Chen (2024)	Node	Graph-Level
Dynamic	Reka et al. (2024)	Temporal	Graph-Level
Dynamic	Okui et al. (2023)	Edge	Edge-Level
Dynamic	Latif et al. (2023)	Node	Edge-Level

the experimental profile of a method (how it was trained and measured) rather than its architectural design, and it supports direct comparison across works.

4.3.1 Performance metrics

Performance metric type refers to the systematic categorization of quantitative measures used to assess the effectiveness and reliability of GNN-based intrusion detection systems, encompassing complementary evaluation dimensions that collectively characterize model performance.

- **Accuracy and F1-Score Group:** Balanced performance indicators that provide comprehensive model assessment.

Accuracy measures overall prediction correctness across all classes and calculated as follows Wang et al. (2023a):

$$Accuracy = \frac{TP + TN}{P + TN + FP + FN}$$

- **F1-Score** is an evaluation metric that combines both precision and recall into a single metric by calculating their harmonic mean. It balances the trade-off between precision (the accuracy of positive predictions) and recall (the ability to detect positives accurately). F1-score is especially valuable in scenarios involving imbalanced data, which are common in intrusion detection tasks (Wang et al. 2023a):

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- **Precision, Recall, and AUC Group:** Quality-focused metrics emphasizing detection capability.

Precision is a crucial metric in intrusion detection systems as it measures the reliability of an IDS in correctly identifying attacks without raising false alarms. High precision indicates fewer false positives, which is essential for real-world applications where excessive false alarms can lead to alert fatigue. The Precision formula (Frenken et al. 2025) is given as,

$$Precision = \frac{TP}{TP + FP}$$

- **Recall** (also called Sensitivity) is an important evaluation metric in intrusion detection, representing the ability of the model to correctly detect actual intrusions. Detection Rate and Recall are essentially the same metric when used in the context of Intrusion Detection Systems (IDS) provides a similar recall formula explicitly as Frenken et al. (2025).

$$Recall = \frac{TP}{TP + FN}$$

- **AUC (area under curve)** is a widely-used evaluation metric for binary classification tasks, reflecting a model's capability to distinguish between positive (malicious) and negative (benign) classes. It summarizes the performance across all classification thresholds. AUC is calculated based on the Receiver Operating Characteristic

(ROC) curve, which plots the True Positive Rate (TPR, also known as recall) against the False Positive Rate (FPR):

$$AUC = \int_0^1 TPR(FPR^{-1}(x)) dx$$

Alternatively, it is commonly presented as the integral of the ROC curve or described as the probability that the classifier ranks a randomly chosen positive instance higher than a randomly

- **False positive rate and detection rate group:** Operational effectiveness metrics.

False Positive Rate (FPR) Measures the proportion of actual negative instances that are incorrectly classified as positive (Frenken et al. 2025).

$$FPR = \frac{FP}{FP + TN}$$

- **Detection rate (DR)** also referred to as True Positive Rate or Recall in binary IDS evaluation, measures the proportion of actual attacks correctly detected by the model:

$$DR = TPR = Recall = \frac{TP}{TP + FN}$$

DR should be reported directly from the confusion matrix, not estimated using dataset-level or architecture-level coefficients. Dataset imbalance, graph construction, learning paradigm, and model architecture can influence DR, but their effects must be measured empirically for each experiment rather than inserted as fixed parameters. In this survey, these factors are therefore treated as explanatory variables for interpreting results, not as components of a generalized DR equation (see Table 6).

4.3.2 Learning paradigm

Learning paradigm defines the fundamental training approach that determines how GNN-based intrusion detection systems utilize labeled and unlabeled data to learn attack patterns and normal network behaviors.

- **Supervised Learning:** A learning framework where the GNN is trained using completely labeled datasets $D = \{(x_i, y_i)\}_{i=1}^n$, where x_i represents graph-structured network data and y_i represents corresponding

Table 6 Standard reporting factors for Detection Rate in GNN-based intrusion detection evaluation

Factor Affecting DR	Why It Matters	How It Should Be Reported
Attack prevalence	Low attack prevalence can make Accuracy misleading	Report DR/Recall, F1-Score, Precision, PR-AUC, and confusion counts
Split strategy	Random splits may leak temporal or repeated-flow patterns	Report random, temporal, or cross-network split explicitly
Graph construction	Endpoint-based graphs may memorize identities; similarity or temporal graphs may generalize better	Report nodes, edges, edge rules, time windows, and leakage controls
Learning paradigm	Supervised, semi-supervised, and self-supervised settings use labels differently	Report label availability and training objective
Thresholding	DR can change substantially under different thresholds	Report fixed threshold, validation-calibrated threshold, or target-FPR threshold
Runtime constraints	Delayed graph construction can make high DR operationally less useful	Report training time, inference time, graph-building overhead, and window size

Detection rate should be computed directly from the confusion matrix as $TP/(TP+FN)$, while dataset properties, graph construction choices, learning paradigms, thresholding, and runtime constraints should be reported as empirical evaluation factors rather than embedded as fixed coefficients

ground-truth labels (normal/attack). The objective function optimizes prediction accuracy based on known labels.

- **Semi-Supervised Learning:** A hybrid approach combining labeled data $DL = \{(x_i, y_i)\}_{i=1}^{n^L}$ and unlabeled data $DU = \{x_i\}_{i=n^L+1}^{n^L+n^U}$, where $n^U \gg n^L$, leveraging both supervised signals and underlying data structure for learning. Semi-supervised GNNs for IDS utilize graph structure to propagate label information across connected network entities.
- **Unsupervised Learning:** A learning paradigm that discovers patterns in network data without explicit labels, typically through techniques such as graph autoencoders, clustering, or anomaly detection methods. For GNN-based IDS, unsupervised approaches identify normal network behavior patterns and detect deviations as potential intrusions.

Supervised approaches require extensive labeled network traffic data, semi-supervised methods leverage abundant unlabeled network flows with limited labeled samples, while unsupervised approaches detect novel attacks without requiring labeled training data. Because these paradigms do not assume the same operational setting, Table 7 summarizes how deployment location, attacker capability, and labeling requirements differ across the main method families discussed in this survey.

4.3.3 Dataset

Dataset refers to the standardized network traffic collections used for training and evaluating GNN-based intrusion detection systems, characterized by specific network environments, attack types, feature sets, and temporal characteristics.

Table 7 Threat-model alignment across IDS method families considered in the survey and empirical case study

Method family	Typical deployment point	Primary attacker assumption	Label requirement	Fairness implication
Supervised tabular/flow classifiers	Sensor, edge collector, SIEM, or backend analytics node	Known attack families are represented in historical labels; evasion occurs mainly through feature manipulation or distribution shift	High-quality labeled benign/attack data	Strong baselines when flow features are already discriminative; should not be dismissed simply because they are non-graph
Sequential deep IDS models	Edge/backend analytics over ordered packets, flows, or sessions	Attacks exhibit temporal signatures or packet/flow ordering effects	Usually labeled sequences, sometimes weak labels	Fair when temporal ordering is available; less comparable to static graph models if sequence information is removed
Endpoint-based GNN IDS	Backend/SIEM or graph analytics layer with host/IP/service identifiers	Attacker behavior is correlated through persistent entities, communication neighborhoods, or lateral movement paths	Labeled nodes, edges, flows, or graph events depending on task	Can exploit relational identity, but must control endpoint memorization and train/test leakage
Endpoint-free or flow-similarity GNN IDS	Edge/backend analytics where privacy or leakage control discourages direct endpoint identifiers	Attacks share behavioral or statistical flow patterns rather than stable host identities	Usually labeled flow records or weak labels	Reduces identity leakage but may lose useful host-context information and can be sensitive to graph-construction choices
Self-supervised/anomaly graph models	Backend analytics, streaming monitor, or periodic retraining pipeline	Unknown, zero-day, or drifting behavior deviates from learned normal relational patterns	Limited or delayed labels; often normal-only or unlabeled training data	Appropriate for label scarcity, but binary supervised metrics may understate or overstate their intended zero-day value

- **CICIDS2017**: A comprehensive intrusion detection dataset created by the Canadian Institute for Cybersecurity, containing labeled network flows with full packet payloads in pcap format. It includes benign traffic and 14 attack categories across 5 days of network activity, with 79 statistical features extracted using CICFlowMeter. The dataset encompasses 2,827,876 records representing realistic network behavior and contemporary attack scenarios (Sharafaldin et al. 2018).
- **UNSW-NB15**: Generated using IXIA PerfectStorm, tcpdump, Argus, and Bro-IDS tools, containing approximately 2,540,044 network flow records with 49 features each. It includes nine attack categories: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms, with an $\frac{87\%}{13\%}$ distribution between normal and malicious traffic (Moustafa and Slay 2016, 2015).
- **BoT-IoT**: Created in UNSW Canberra's Cyber Range Lab, representing a realistic IoT network environment with 62 hosts. The dataset contains over 72 million records with 46 features, including DDoS, DoS, reconnaissance, and information theft attacks. The extracted 5% subset comprises approximately 3 million records across four main attack categories (Koroniotis et al. 2019).

These datasets provide standardized evaluation benchmarks for GNN-based IDS systems, with CICIDS2017 offering comprehensive feature sets, UNSW-NB15 providing diverse modern attack types, and BoT-IoT focusing on IoT-specific network environments and attack patterns.

At the same time, the field is gradually moving beyond these legacy flow-based benchmarks toward newer temporal, multi-entity, and graph-oriented datasets. Recent examples include CICIOT2023 and emerging CIC graph-oriented malware datasets such as CIC-DGG and CIC-SGG, which are intended to expose richer temporal dependencies, device-level interactions, software-defined-network behavior, and graph-structured security relations (Neto et al. 2023; Shokouhinejad et al. 2025; Mohammadian et al. 2025). These datasets are important for future GNN-IDS evaluation because they can test whether message passing, temporal aggregation, heterogeneous graph modeling, and graph-level attack reasoning provide benefits that are not visible in static flow tables. However, they are not yet as uniformly adopted as CICIDS2017, UNSW-NB15, or BoT-IoT; therefore, cross-paper comparison remains difficult unless authors report graph construction rules, temporal windows, node and edge definitions, label granularity, and graph construction overhead in addition to standard detection metrics.

This mapping (Table 8) highlights that a GNN architecture should not be selected independently of dataset structure. GraphSAGE-style sampling is attractive for large graphs because it limits neighborhood expansion, whereas attention-based and edge-aware models are more useful when edge attributes carry the security signal. Temporal GNNs are preferable when the dataset preserves event ordering or sliding-window traffic dynamics. Self-supervised and pre-trained GNNs become more relevant when labeled attack data is scarce or when models must adapt across networks. Recent practical GNN work emphasizes that realistic deployment requires few-second temporal windows, unlabeled pre-training, and reduced dependence on target-network labels (PPT-GNN Van Langendonck et al. 2025).

Subsequently, Table 9 summarizes the representative studies evaluated according to aforementioned characteristics.

4.4 Insights and trends

By analyzing the research works on Graph Neural Networks (GNNs) for network security, several key insights and trends across different aspects can be derived.

4.4.1 Investigated intrusion types and their distribution

The most frequent category is network intrusion and anomalies, addressed in over 50% of surveyed papers. This category encompasses a broad spectrum of threats, including Distributed Denial of Service (DDoS) attacks,

Table 8 Dataset-method mapping for GNN-based IDS evaluation, summarizing the main characteristics of common IDS dataset families, suitable graph-construction strategies, appropriate GNN families, and key fairness risks that can affect comparative evaluation

Dataset family	Typical characteristics	Suitable graph construction	Suitable GNN families	Main fairness risk
UNSW-NB15 (Moustafa and Slay 2016, 2015)	Enterprise-style flow records with mixed attack categories and moderate imbalance	Flow-to-flow similarity graphs, endpoint-free graphs, edge-feature graphs	Edge-aware GraphSAGE variants, endpoint-free flow-graph GNNs, explainable graph-based botnet detectors, and GCN-based attack detectors	Duplicate flows, temporal leakage, endpoint memorization
CICIDS2017 / CSE-CIC-IDS (Sharafaldin et al. 2018)	Rich labeled flow scenarios with multiple attack days	Host-flow graphs, flow similarity graphs, temporal windows	GCN, GAT, GraphSAGE, temporal GNNs	Random splits can mix near-duplicate temporal behavior
BoT-IoT (Koroniotis et al. 2019)/NF-BoT-IoT (Sarhan et al. 2021)	IoT-heavy botnet and DDoS traffic with severe imbalance	Edge-centric and scalable sampled graphs	Sampling-based GNNs, edge-aware GraphSAGE variants, attention-based GNNs, and spatio-temporal GNNs	Dominant attack families can inflate aggregate accuracy
ToN-IoT (Moustafa 2021)/NF-ToN-IoT (Sarhan et al. 2021)	Heterogeneous telemetry, IoT, network, and system signals	Heterogeneous and temporal graphs	Heterogeneous GNNs, temporal GNNs, self-supervised GNNs	Cross-source heterogeneity complicates direct comparison
LANL (Kent 2015)/Provenance-based (Five Directions 2020; DARPA 2018; Cai et al. 2026)/BETH-style logs (Highnam et al. 2021)	Authentication, process, or event sequences with temporal dependencies	Provenance graphs, user-host graphs, temporal event graphs	TGN, temporal GAT, provenance GNNs, graph autoencoders	Label sparsity and delayed attack manifestation

malware infections, botnet activities, and insider threats. Another significant category is IoT security attacks, constituting approximately 20% of the reviewed studies. Researchers have particularly emphasized attacks analyzed through the BoT-IoT dataset, reflecting a growing concern with securing IoT environments due to their inherent vulnerabilities and widespread adoption. False Data Injection (FDI) attacks also emerge as a critical area of study, especially prominent within research focused on smart grids and Industrial IoT (IIoT) systems. The substantial presence of FDI-related studies indicates a rising awareness of threats posed by manipulated data in critical infrastructure and industrial operations. In recent years, studies examining web-based attacks have shown an increasing trend, currently making up around 10% of the surveyed literature. These attacks include Cross-Site Scripting (XSS), web shells, and malicious HTTP payloads, highlighting the evolving complexity and persistence of threats targeting web applications and servers. Finally, research into vehicle network attacks, specifically focusing on Controller Area Network (CAN) bus systems and automotive intrusion detection, represents a rapidly growing segment. This growth aligns with rising security concerns associated with the increasing digitization and connectivity of smart vehicles, underscoring the importance of securing automotive communication infrastructures.

As shown in Fig. 6, Network Intrusion and IoT Security dominate research (~61%). Web security and industrial threats (FDI, VAN, Smart Grid) are growing focus areas. IoT and Smart Grid Security are rapidly growing domains where GNN-based solutions are being actively researched.

4.4.2 Deployment environments

Graph Neural Networks (GNNs) have been successfully applied across diverse networking environments, each presenting distinct challenges and security requirements. Traditional and generic computer networks represent the most frequent application domain, comprising over 60% of surveyed research (see Fig. 7). Studies primarily

Table 9 Evaluation methodologies adopted in GNN-based IDS studies, grouping works by primary performance metrics (accuracy/F1-score, precision-recall-AUC, or false-positive and detection rates) and summarizing for each reference the learning paradigm (supervised, semi-supervised, or unsupervised) and benchmark dataset used (e.g., CICIDS2017, UNSW-NB15, BoT-IoT, ToN-IoT, BETH or custom corpora), thereby highlighting prevailing evaluation practices and revealing underexplored combinations of metrics, learning regimes, and datasets in the current literature

Performance metric type	References	Learning paradigm	Dataset
Accuracy, F1-Score	Bilot et al. (2023a)	Supervised	CICIDS2017
Accuracy, F1-Score	Singh et al. (2024)	Supervised	UNSW-NB15
Accuracy, F1-Score	Le and Park (2024)	Semi-supervised	CICIDS2017
Accuracy, F1-Score	Tang et al. (2022)	Semi-supervised	BETH
Accuracy, F1-Score	Lakshmanan et al. (2024)	Supervised	BETH
Accuracy, F1-Score	Wei et al. (2023)	Supervised	BETH
Accuracy, F1-Score	Das (2024)	Semi-supervised	BETH
Accuracy, F1-Score	Takiddin et al. (2023a)	Supervised	BETH
Accuracy, F1-Score	Li et al. (2023b)	Supervised	BETH
Accuracy, F1-Score	Boyaci et al. (2022)	Supervised	BETH
Accuracy, F1-Score	Abinesh et al. (2024)	Supervised	UNSW-NB15
Accuracy, F1-Score	Deng and Huang (2025)	Semi-Supervised	UNSW-NB15
Accuracy, F1-Score	Han et al. (2024)	Supervised	UNSW-NB15
Accuracy, F1-Score	Pujol-Perich et al. (2022)	Supervised	CICIDS2017
Accuracy, F1-Score	Lo et al. (2022)	Semi-Supervised	ToN-IoT
Accuracy, F1-Score	Manjula et al. (2023)	Supervised	BoT-IoT
Accuracy, F1-Score	Kumar and Khari (2025)	Supervised	UNSW-NB15
Accuracy, F1-Score	Park et al. (2023)	Supervised	UNSW-NB15
Accuracy, F1-Score	Peng and Zhang (2024)	Supervised	ToN-IoT
Accuracy, F1-Score	Zhou et al. (2020)	Supervised	CICIDS2017
Accuracy, F1-Score	Marfo et al. (2024)	Supervised	UNSW-NB15
Accuracy, F1-Score	Casas et al. (2023)	Supervised	CICIDS2017
Accuracy, F1-Score	Roy and Chen (2024)	Semi-Supervised	ToN-IoT
Accuracy, F1-Score	Shi et al. (2021)	Supervised	UNSW-NB15
Accuracy, F1-Score	Xu et al. (2023)	Supervised	UNSW-NB15
Accuracy, F1-Score	Huang et al. (2022)	Unsupervised	CICIDS2017
Accuracy, F1-Score	Xu et al. (2024a)	Semi-supervised	CICIDS2017
Accuracy, F1-Score	Hassan et al. (2023)	Supervised	CICIDS2017
Accuracy, F1-Score	Liu et al. (2018)	Supervised	Custom Dataset
Precision, Recall, AUC	Lv et al. (2023)	Supervised	BETH
Precision, Recall, AUC	Caville et al. (2022)	Unsupervised	CICIDS2017
Precision, Recall, AUC	Venturi et al. (2023)	Unsupervised	CICIDS2017
Precision, Recall, AUC	Chang and Branco (2024)	Supervised	UNSW-NB15
Precision, Recall, AUC	Lakha et al. (2022)	Supervised	BETH
Precision, Recall, AUC	Saunders et al. (2024)	Supervised	BoT-IoT
Precision, Recall, AUC	Chen et al. (2023)	Supervised	BETH
Precision, Recall, AUC	Takiddin et al. (2023b)	Supervised	ToN-IoT
Precision, Recall, AUC	Zhang et al. (2023)	Supervised	UNSW-NB15
Precision, Recall, AUC	Cuong et al. (2023)	Supervised	UNSW-NB15
Precision, Recall, AUC	Maksimski (2023)	Semi-Supervised	BoT-IoT
Precision, Recall, AUC	Huang et al. (2023)	Supervised	CICIDS2017
Precision, Recall, AUC	Ji and Huang (2022)	Supervised	CICIDS2017
Precision, Recall, AUC	Wang et al. (2023c)	Supervised	UNSW-NB15
Precision, Recall, AUC	Farrukh et al. (2025)	Supervised	CICIDS2017
Precision, Recall, AUC	Ding et al. (2024)	Supervised	BoT-IoT
Precision, Recall, AUC	Mirlashari and Rizvi (2024)	Semi-Supervised	BoT-IoT
Precision, Recall, AUC	Purnama et al. (2022)	Supervised	BoT-IoT
Precision, Recall, AUC	Messai and Seba (2023)	Semi-Supervised	BoT-IoT
Precision, Recall, AUC	Friji et al. (2023)	Supervised	BoT-IoT
Precision, Recall, AUC	Boukari et al. (2024)	Semi-Supervised	BoT-IoT

Table 9 (continued)

Performance metric type	References	Learning paradigm	Dataset
Precision, Recall, AUC	Bajpai et al. (2024)	Semi-Supervised	UNSW-NB15
Precision, Recall, AUC	Gharavian et al. (2023)	Supervised	CICIDS2017
Precision, Recall, AUC	Anoop et al. (2024)	Supervised	UNSW-NB15
Precision, Recall, AUC	Bora and Chauhan (2023)	Supervised	ToN-IoT
Precision, Recall, AUC	Manikandan et al. (2024)	Supervised	ToN-IoT
Precision, Recall, AUC	Wu et al. (2024)	Semi-Supervised	ToN-IoT
Precision, Recall, AUC	Chen et al. (2023)	Semi-supervised	CICIDS2017
Precision, Recall, AUC	Pan et al. (2024)	Supervised	ToN-IoT
Precision, Recall, AUC	Bao et al. (2023)	Supervised	UNSW-NB15
Precision, Recall, AUC	Xu et al. (2023)	Semi-Supervised	ToN-IoT
Precision, Recall, AUC	Zoubir and Missaoui (2025)	Semi-supervised	CICIDS2017
Precision, Recall, AUC	Xiao et al. (2023)	Supervised	Custom Dataset
Precision, Recall, AUC	Okui et al. (2023)	Semi-Supervised	UNSW-NB15
Precision, Recall, AUC	Long et al. (2023)	Supervised	ToN-IoT
Precision, Recall, AUC	Latif et al. (2023)	Unsupervised	CICIDS2017
Precision, Recall, AUC	Wang et al. (2023b)	Unsupervised	Custom Dataset
Precision, Recall, AUC	Sharma et al. (2022)	Supervised	Custom Dataset
Precision, Recall, AUC	Ferretti et al. (2024)	Supervised	Custom Dataset
False-Positive Rate, Detection Rate	Saunders et al. (2024)	Supervised	BoT-IoT
False-Positive Rate, Detection Rate	Liu et al. (2021)	Supervised	BoT-IoT
False-Positive Rate, Detection Rate	Barsellotti et al. (2023)	Supervised	CICIDS2017
False-Positive Rate, Detection Rate	Li et al. (2022)	Supervised	CICIDS2017
False-Positive Rate, Detection Rate	Ren and Geng (2025)	Supervised	UNSW-NB15
False-Positive Rate, Detection Rate	Alizadeh and Khansari (2023)	Unsupervised	CICIDS2017
False-Positive Rate, Detection Rate	Gorricho-Segura et al. (2023)	Supervised	CICIDS2017
False-Positive Rate, Detection Rate	Cheng et al. (2024)	Supervised	CICIDS2017
False-Positive Rate, Detection Rate	Wei et al. (2023)	Supervised	BoT-IoT
False-Positive Rate, Detection Rate	Biswas et al. (2023)	Semi-Supervised	UNSW-NB15
False-Positive Rate, Detection Rate	Song et al. (2024)	Supervised	UNSW-NB15
False-Positive Rate, Detection Rate	El-Dawy (2024)	Supervised	UNSW-NB15
False-Positive Rate, Detection Rate	Xu et al. (2024b)	Supervised	BoT-IoT
False-Positive Rate, Detection Rate	Wang et al. (2023a)	Supervised	UNSW-NB15

Fig. 6 Distribution of intrusion types addressed in GNN-based IDS literature, showing that Network Intrusion and Anomalies constitute the dominant focus (41 studies), followed by IoT-specific attacks (20) and False Data Injection in critical infrastructure (12), while DDoS (9), web application attacks (8), vehicular network attacks (6), and other attack types (4) represent emerging but comparatively underexplored research directions

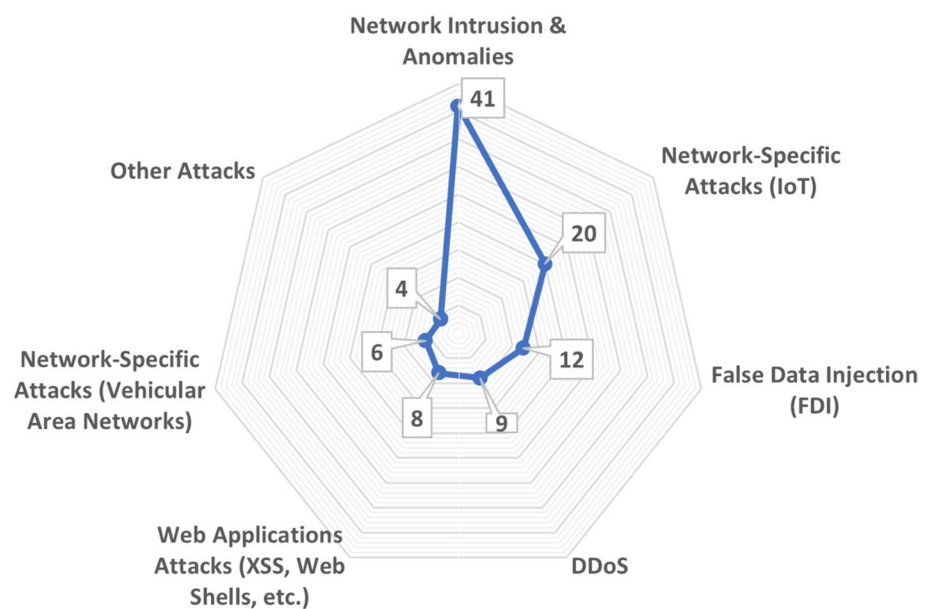
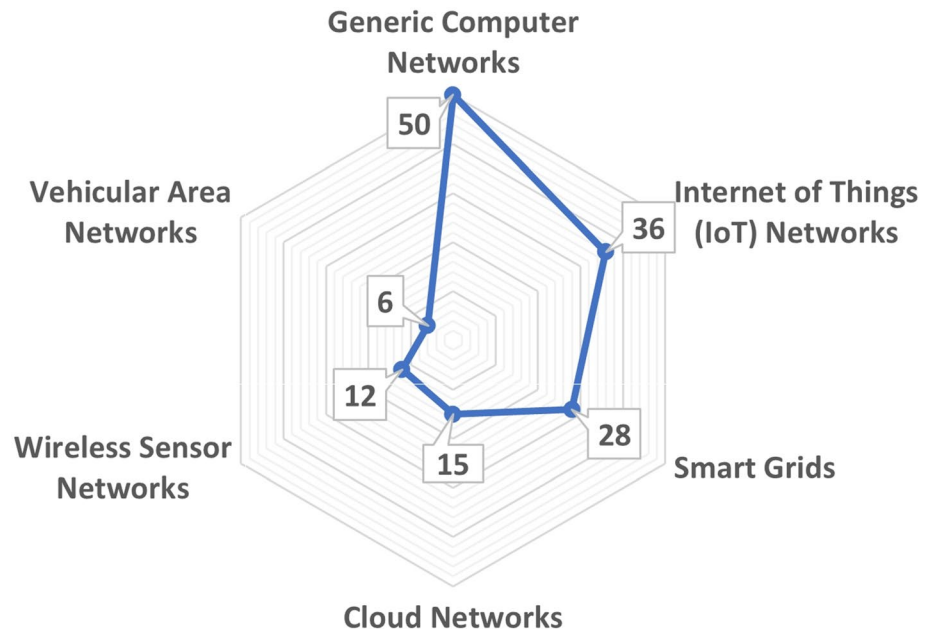


Fig. 7 Network environments targeted by GNN-based IDS research, indicating that generic computer networks are the primary focus (50 studies), followed by Internet of Things (36) and smart grid infrastructures (28), while cloud networks (15), wireless sensor networks (12), and vehicular area networks (6) constitute smaller but rapidly emerging domains for graph-based intrusion detection



target enterprise network security and backbone infrastructures, reflecting the continued emphasis on protecting conventional IT environments from advanced cyber threats such as malware, botnets, and insider attacks. The domain of Internet of Things (IoT) and Industrial IoT (IIoT) constitutes a significant portion -over 30%- of the reviewed literature. Research focused on cloud security is gradually gaining attention, accounting for approximately 10% of recent studies. This growing area particularly emphasizes the use of federated learning methods to enhance privacy-preserving intrusion detection within cloud-based infrastructures and multi-party systems. Finally, Wireless Sensor Networks (WSN) represent an emerging research area, currently constituting about 8% of the reviewed literature. With the rapid expansion of wireless and mobile network deployments, this domain is expected to grow substantially, driven by the need to secure the next generation of connectivity infrastructures from sophisticated cyber threats.

IoT and IIoT security research is expanding (~36%), highlighting the need for specialized GNN models. Smart grids and vehicle security are still emerging but have significant security implications. IIoT and Smart Grid Security are emerging as key areas for graph-based anomaly detection, especially for real-time attack detection.

Practical IDS deployment requires reporting more than classifier-level metrics. In other words, deployment feasibility depends on more than classifier accuracy. A practical GNN-IDS must specify how packets or logs are collected, how flows are aggregated, how graph nodes and edges are updated, how long a detection window is allowed to remain open, and whether graph construction runs on the sensor, switch, edge server, or central backend. FlowLens demonstrates the value of compact packet-distribution markers for efficient ML-based flow classification at line speed, while NetBeacon shows how intelligent data-plane design can reduce control-plane pressure under explicit memory and processing constraints (Barradas et al. 2021; Zhou et al. 2023). EdgeTorrent similarly frames real-time intrusion detection as an end-to-end temporal graph representation problem rather than a classifier-only problem (King et al. 2023). For GNN-IDS evaluation, this means reporting graph construction time, edge-list size, window duration, update frequency, inference latency, and memory footprint alongside Accuracy, F1-Score, Detection Rate, Recall, Precision, and AUC.

The main deployment challenges can be grouped into five categories. First, graph construction can dominate total latency when edges are induced from pairwise similarity, temporal co-occurrence, provenance paths, or host-flow interactions; therefore, edge-list growth should be reported as a function of the number of flows, endpoints, and time-window length. Second, encrypted traffic reduces payload visibility and shifts the burden toward metadata, timing, size, and behavioral features, which may favor endpoint-free or sequence-aware representations but

also increases ambiguity. Third, adversarially adapted traffic can manipulate features, timing, or graph structure, so robustness should be evaluated under realistic evasion and poisoning assumptions rather than inferred from clean benchmark accuracy. Fourth, memory and data-plane constraints determine whether graph construction and inference can run close to the sensor or must be moved to an edge or backend layer. Fifth, static-graph assumptions become stale in streaming traffic: a graph snapshot that is accurate for one time window may misrepresent later behavior under concept drift, topology changes, or attacker migration. These constraints explain why the quantitative section reports timing and stability together with classifier metrics, and why static public-dataset results should be interpreted as benchmark evidence rather than direct proof of production readiness.

4.4.3 Feature encoding trends

The choice of features utilized in Graph Neural Network (GNN) models significantly influences their performance in intrusion detection applications. These features can be categorized broadly into node features, edge features, and temporal features, each playing distinct roles in capturing different aspects of network behavior.

Node Features primarily characterize individual entities within the network graph. Among these:

- **Traffic flow and IP address behavior** features are widely employed in studies focusing on IoT security and network anomaly detection. Such features help in identifying unusual patterns and potential threats within large-scale network traffic datasets.
- **Device behavior and communication logs** are commonly leveraged in industrial networks and automotive security, where understanding the standard operational patterns of devices or vehicles is essential for identifying deviations indicative of intrusions or malicious activity.

Edge Features capture relationships or interactions between entities and include:

- **Packet relations and protocol interactions**, utilized in approximately 30% of studies, which offer insights into packet-level communication and the protocols governing interactions within the network.
- **Connection types and network traffic patterns**, widely adopted (around 50% of papers), are crucial for capturing how entities communicate across the network. Such features are instrumental in identifying anomalies and understanding the nature of different types of attacks, particularly in network-level security assessments.

Temporal Features provide insights into dynamic changes occurring within networks over time, including:

- **User activity and behavioral changes**, featured in about 20% of reviewed literature, helping detect intrusions based on deviations from established user or device patterns over specific time intervals.
- **Power consumption patterns**, appearing in around 10% of papers focusing on smart grid security. These features are particularly effective for identifying anomalies that could indicate false data injection attacks or other manipulations within critical infrastructure systems.

Traffic flow analysis is the dominant feature in most research. Temporal features are becoming more important, especially in behavior-based anomaly detection. More papers are integrating edge features and temporal analysis for better real-time detection. A summary of the findings is given in Table 10.

Table 10 Feature encoding trends in GNN-based IDS, indicating that node-level traffic flow and IP address features are the dominant choice for modeling network and IoT intrusions, while protocol- and packet-oriented edge features are increasingly integrated to capture interaction patterns, and temporal encodings of user activity or behavioral changes are emerging as key signals for behavior-based anomaly detection in smart grid and enterprise environments

Feature type	Common In
Node features: traffic flow & IP addresses	Network intrusion, IoT security
Edge features: protocol relations & packet metadata	IoT, malware detection
Temporal features: user activity logs & behavioral changes	Smart Grid, enterprise security

4.4.4 Learning paradigms

As depicted in Fig. 8, supervised learning is the predominant paradigm, appearing in approximately 70% of reviewed studies. GNN models using supervised approaches typically rely on widely recognized labeled datasets, such as CICIDS2017, UNSW-NB15, and BoT-IoT. The abundance of labeled network security data facilitates model training for accurate identification and classification of known attack types. Semi-supervised learning constitutes about 20% of the literature, utilized primarily for scenarios where labeled data is scarce or costly to obtain. This approach is especially valuable in few-shot learning contexts, anomaly detection tasks, and scenarios requiring unsupervised extraction of meaningful features from partially labeled datasets. Unsupervised learning paradigms are featured in roughly 10% of surveyed research, frequently in the form of self-supervised learning. These methods leverage intrinsic data structures and relationships to detect anomalies without explicit labeling, making them particularly suited for discovering novel and evolving intrusion patterns. Finally, although relatively rare (less than 5%), reinforcement learning approaches have emerged in some recent studies. These works typically explore adaptive intrusion detection systems capable of dynamically optimizing detection strategies through trial-and-error interactions within the network environment, demonstrating the potential for highly adaptive and responsive intrusion detection capabilities.

Supervised learning dominates (~68%), but semi-supervised and unsupervised models are emerging for few-shot learning and anomaly detection. Reinforcement learning is rare (~4%), used mainly in adaptive intrusion detection models.

Evaluation Metrics and Datasets

The majority of studies in graph neural network-based intrusion detection systems (IDSs) report their performance using well-established classification metrics. Accuracy, precision, recall, and F1-score are the most commonly employed, appearing in approximately 90% of the reviewed papers (see Fig. 9), reflecting their utility in evaluating both overall effectiveness and class-wise balance in detection tasks. In addition, metrics such as the false positive rate (FPR) and detection rate (DR) -also referred to as true positive rate or sensitivity- are reported in nearly 60% of the papers, particularly in works focused on operational feasibility and real-time intrusion detection. These metrics are crucial in assessing the practical deployability of IDS models, where minimizing false alarms and maximizing detection efficiency are key priorities. Approximately 30% of the papers, primarily those focused on botnet detection and general anomaly detection, utilize the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) as a metric. The AUC provides a comprehensive view of a model's ability to discriminate between attack and benign traffic across varying classification thresholds, making it especially valuable in imbalanced or uncertain detection scenarios.

Fig. 8 Learning paradigms used in GNN-based IDS studies, indicating a clear dominance of supervised learning (68 works), with semi-supervised (20) and unsupervised approaches (8) forming smaller but important streams that address limited-label and anomaly-detection settings and highlight the need for more label-efficient and self-supervised graph learning methods in intrusion detection

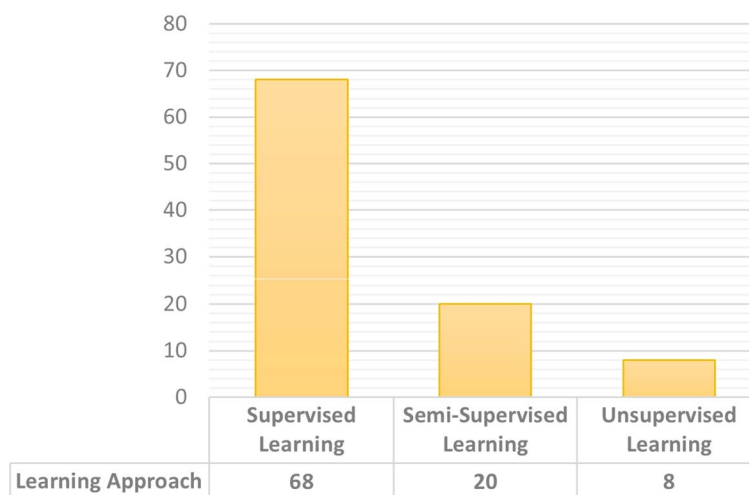
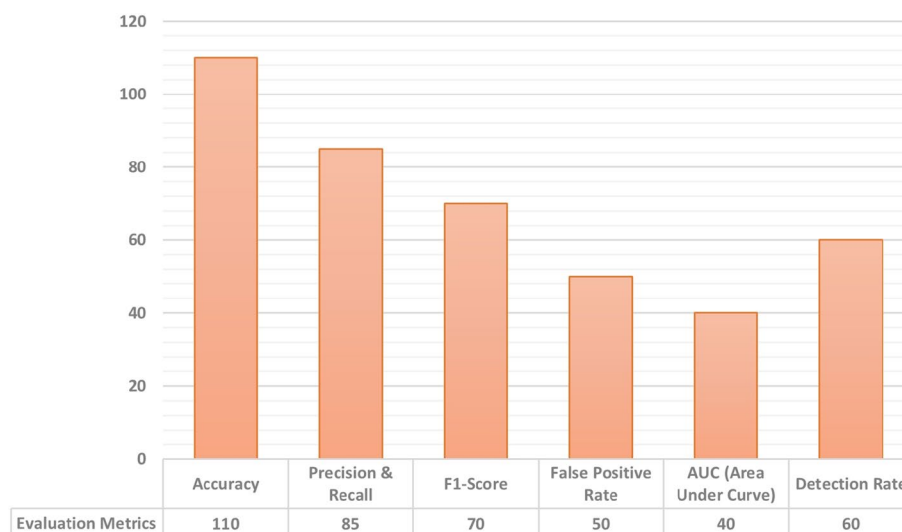


Fig. 9 Evaluation metrics reported in GNN-based IDS studies, showing that accuracy is used most frequently (110 papers), followed by precision and recall (85), F1-score (70), detection rate or sensitivity (60), false positive rate (50), and AUC-ROC (40), underscoring the community's emphasis on both overall classification performance and operational reliability in intrusion detection



Accuracy and F1-score are the most used metrics while AUC (Area Under Curve) is mainly used in botnet and anomaly detection models.

A variety of publicly available datasets have emerged as standard benchmarks for evaluating the effectiveness of Graph Neural Network (GNN) approaches in intrusion detection research. Among these, several datasets are particularly prominent:

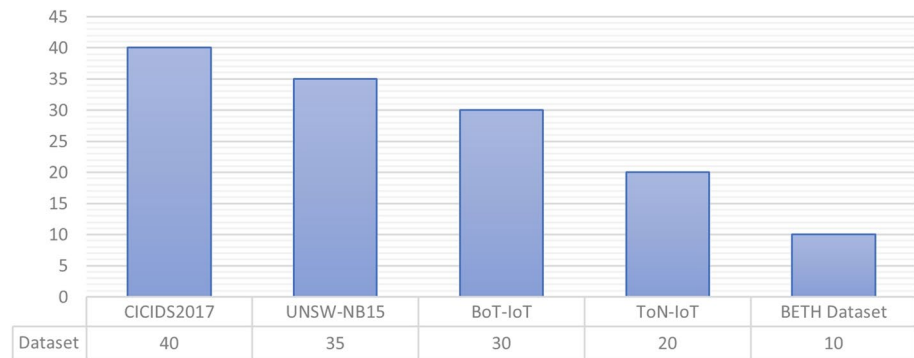
- The CICIDS2017 dataset is the most frequently employed, utilized in approximately 40% of reviewed studies. It is primarily applied in network intrusion detection tasks due to its comprehensive representation of modern attack patterns and realistic benign traffic scenarios.
- The UNSW-NB15 dataset, featured in about 35% of papers, is commonly used for evaluating models in general network anomaly detection. Its diverse attack scenarios and balanced distribution make it ideal for assessing anomaly detection performance in various network contexts.
- The BoT-IoT dataset is widely adopted, appearing in roughly 30% of studies focused specifically on IoT security. It includes scenarios typical to IoT environments, such as botnet attacks, denial-of-service attacks, and malicious payload injections.
- The ToN-IoT dataset, used in approximately 20% of the literature, is particularly valuable for evaluating models in Industrial IoT (IIoT) security contexts. It provides data representative of realistic industrial network operations, including a variety of sophisticated cyber-attacks.
- Finally, the BETH dataset is referenced in around 10% of studies, primarily for research focused on cybersecurity within Smart Grid infrastructures. This dataset offers specialized scenarios relevant to energy networks, highlighting vulnerabilities and attack methods specific to this critical domain.

A summary of the findings is depicted in Fig. 10.

5 Quantitative performance analysis

This section evaluates ten representative intrusion-detection models on a leakage-controlled UNSW-NB15 raw-flow experiment. The experiment uses 360,000 raw records before deduplication. After duplicate flow-label signatures are removed, the modelling set contains 232,803 records. The chronological split assigns 139,682 records to the fixed training pool and 93,121 records to the fixed test set. The primary hard-evaluation slice contains 92,812 records with an attack prevalence of 5.49%. Duplicate control is performed before temporal splitting by constructing a flow-label signature from the available non-endpoint flow attributes and the binary label; only the

Fig. 10 Benchmark datasets used in GNN-based IDS literature, indicating that CICIDS2017 is the most prevalent (40 studies), followed by UNSW-NB15 (35) and BoT-IoT (30), while ToN-IoT (20) and the BETH smart-grid dataset (10) form smaller yet important benchmarks for evaluating graph-based intrusion detection across conventional, IoT, industrial, and energy-sector networks



first occurrence of each signature is retained. Consequently, identical signatures that recur at later timestamps cannot appear simultaneously in the training and test partitions. This setting is stricter than a random split because it reduces repeated-flow effects, evaluates models on a later traffic segment, and avoids over-interpreting duplicate or near-duplicate flows. The experiments were executed on a single-GPU workstation with an NVIDIA GeForce RTX 4060 GPU and 8 GB of dedicated memory. Because runtime measurements depend on hardware, the reported training, inference, and total runtime values are used primarily for within-study comparison under a fixed execution environment.

UNSW-NB15 is used because it is a widely adopted public benchmark for network intrusion detection and because its raw flow-level records, binary labels, attack categories, and heterogeneous traffic attributes support both conventional machine-learning models and graph-based IDS methods (Moustafa and Slay 2015, 2016). The 360,000-record input setting increases the stability of repeated evaluation while keeping the experiment computationally feasible on the available single-GPU environment. The test set is kept fixed across all repeats, and only the training subset is varied. This design makes the reported variability reflect training-sample sensitivity rather than changes in the test population.

The comparison includes three conventional or non-graph baselines, RandomForest-SU, RNN-IDS, and CNN-LSTM; one anomaly-oriented model, Anomal-E; and six graph-based methods drawn from IDS literature, namely Alert-GCN, E-GraphSAGE, FTG-Net, GNN-IDS, NetworkAttack-GCN, and XG-BoT. These implementations were selected systematically to cover the main detector families discussed in the survey rather than to optimize a single benchmark result. RandomForest-SU represents strong tabular tree-ensemble IDS; RNN-IDS and CNN-LSTM represent sequential deep-learning baselines; Anomal-E represents anomaly-oriented detection; and the six graph-based methods cover alert graphs, GraphSAGE-style neighborhood aggregation, hierarchical flow-to-traffic graphs, generic GNN-IDS pipelines, GCN-based attack detection, and explainable botnet-oriented graph learning (Lo et al. 2022; Caville et al. 2022; Le and Park 2024; Sun et al. 2024; Lo et al. 2023; Barsellotti et al. 2023; Abdullayeva and Suleymanzade 2024; Cheng et al. 2021). The selection therefore balances architectural diversity, IDS relevance, implementation feasibility on the same hardware, and fair comparison against non-GNN alternatives. Conventional baselines are included because tree ensembles and deep sequence models are still common in practical IDS pipelines and provide necessary reference points for judging when graph message passing adds operational value.

For the graph-based representatives, the endpoint-free topology is constructed without using raw source IP, destination IP, source port, or destination port as direct node identifiers. Instead, flow records are treated as traffic events and connected through non-endpoint behavioral similarity and temporal compatibility derived from normalized flow attributes. This avoids creating trivial host-identity shortcuts and reduces the risk that the model memorizes repeated endpoint pairs rather than learning transferable traffic behavior. Node features correspond to normalized flow attributes, edge construction uses the retained non-endpoint representation, and binary labels are assigned at the flow/event level. Timing values in Sect. 5 include model training and model inference under the

implemented pipeline; where graph construction is required, its contribution is included in total processing time for the graph-based implementation rather than treated as a free preprocessing step.

5.1 Repeated-evaluation protocol and statistical testing

To maintain statistical reliability, each model is trained and evaluated 30 times. In each repeat, 80% of the fixed training pool is drawn as a stratified training subset, while the test set and the hard-evaluation mask remain unchanged. This yields 30 paired observations per model and metric under the same test distribution. The reported intervals are 95% confidence intervals around the repeated-run mean. These intervals quantify sensitivity to training-subsample variation under one fixed temporal test set; they should not be interpreted as proof of generalization to all future network traffic. Tight intervals can occur when the fixed test distribution is stable and the model ranking is consistent, but external validity still requires additional streaming, cross-dataset, and concept-drift evaluations. Because the same set of ten models is compared across the same 30 repeated experimental blocks, the Friedman rank test is used as the primary omnibus test for multi-model comparison (Demšar 2006). Paired t-tests and Wilcoxon signed-rank tests are also computed as supplementary pairwise checks against the best model for each metric.

The Friedman tests reject equality of model ranks for all reported metrics, with $p < 0.001$ for Accuracy, F1-Score, Precision, Detection Rate/Recall, AUC, training time, inference time, and total time. This result is expected when model ranks are highly consistent across repeated experimental blocks. The full numerical p-values are retained in the experimental output files, while the manuscript reports the thresholded result to avoid unnecessary false precision.

5.2 Accuracy and F1-score

Figure 11 reports Accuracy with repeated-run confidence intervals. RandomForest-SU achieves the highest mean Accuracy at 0.9797. CNN-LSTM and XG-BoT follow with 0.9762 and 0.9745, respectively, while NetworkAttack-GCN obtains 0.9728. The graph-based models therefore do not dominate Accuracy; instead, the strongest tabular baseline is preferable when the primary objective is aggregate benign-versus-attack correctness.

The F1-Score comparison in Fig. 12 is more balanced. CNN-LSTM obtains the highest mean F1-Score at 0.8221, closely followed by RandomForest-SU at 0.8215 and XG-BoT at 0.8116. NetworkAttack-GCN reaches 0.8018, while E-GraphSAGE reaches 0.7661. This pattern indicates that graph-based models are competitive, but the class-imbalance trade-off between Precision and Detection Rate still determines their F1 ranking.

Fig. 11 Accuracy of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals with a fixed hard-evaluation test set; the y-axis is zoomed to improve readability

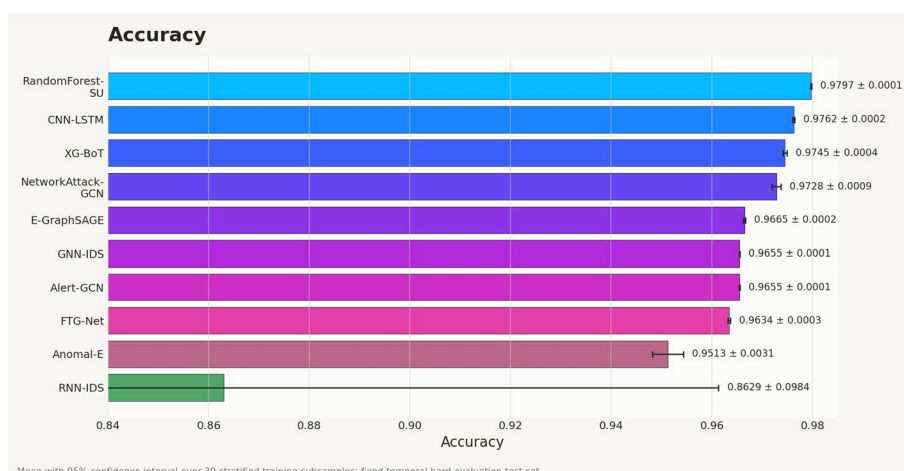


Fig. 12 F1-Score of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals; the y-axis is zoomed to emphasize between-model differences

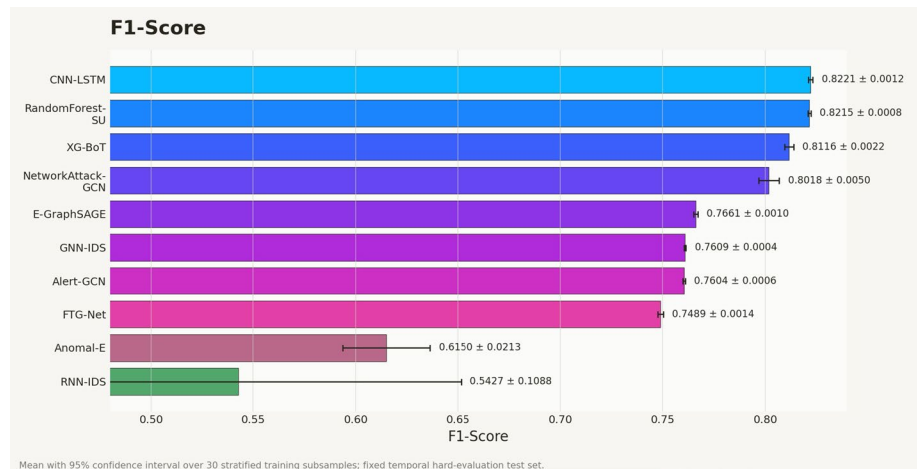
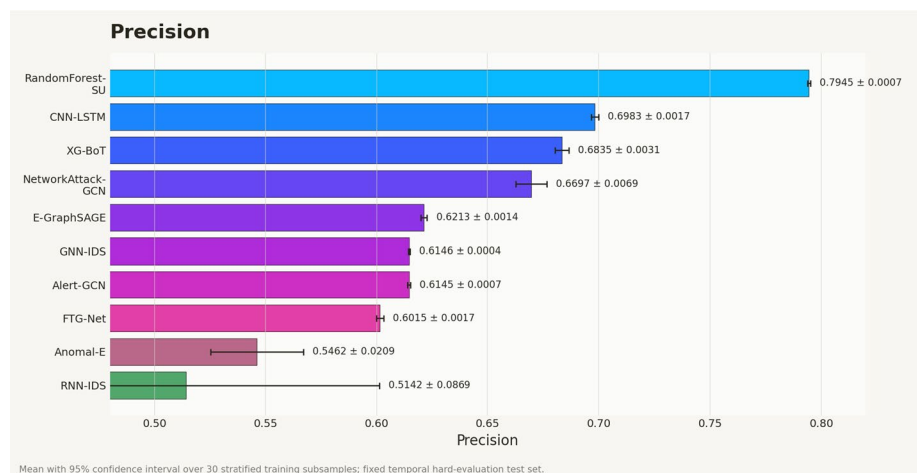


Fig. 13 Precision of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals; the y-axis is zoomed to improve visual separation



5.3 Precision, detection rate, and recall

Precision is the clearest advantage of the strongest conventional baseline. As shown in Fig. 13, RandomForest-SU reaches a mean Precision of 0.7945, while CNN-LSTM reaches 0.6983 and XG-BoT reaches 0.6835. Several graph models mark more flows as attacks, which increases false positives but substantially reduces missed attacks.

Detection Rate shows a different operational pattern. Figure 14 shows that NetworkAttack-GCN reaches a mean Detection Rate of 0.9993, XG-BoT reaches 0.9990, GNN-IDS reaches 0.9988, E-GraphSAGE reaches 0.9987, CNN-LSTM reaches 0.9992, and Alert-GCN reaches 0.9971. RandomForest-SU, despite its superior Precision and Accuracy, detects 85.04% of attacks on average. For practical IDS deployment, this gap matters because missed attacks are often more costly than a moderate increase in analyst workload.

Recall mirrors Detection Rate because both measure the proportion of true attacks correctly detected in this binary setting. Figure 15 therefore confirms the same conclusion: the graph-based group, together with CNN-LSTM, is substantially more sensitive to attacks than RandomForest-SU and Anomal-E.

5.4 AUC and ranking stability

The AUC comparison in Fig. 16 favors RandomForest-SU, which achieves a mean AUC of 0.9954. CNN-LSTM, XG-BoT, and NetworkAttack-GCN are close with 0.9926, 0.9924, and 0.9919, respectively. E-GraphSAGE

Fig. 14 Detection rate of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals; the y-axis is zoomed to highlight high-recall behavior

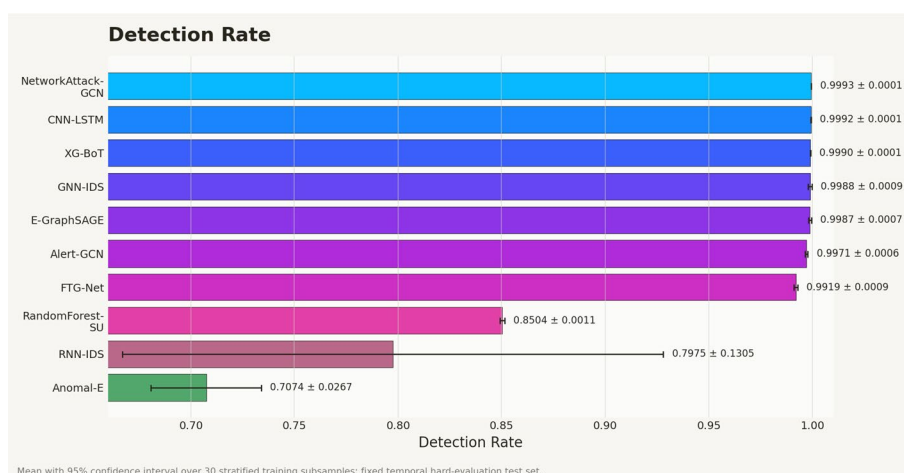


Fig. 15 Recall of the ten evaluated IDS models over 30 repeated training subsamples. Recall is numerically equivalent to Detection Rate in the binary evaluation setting; the y-axis is zoomed for readability

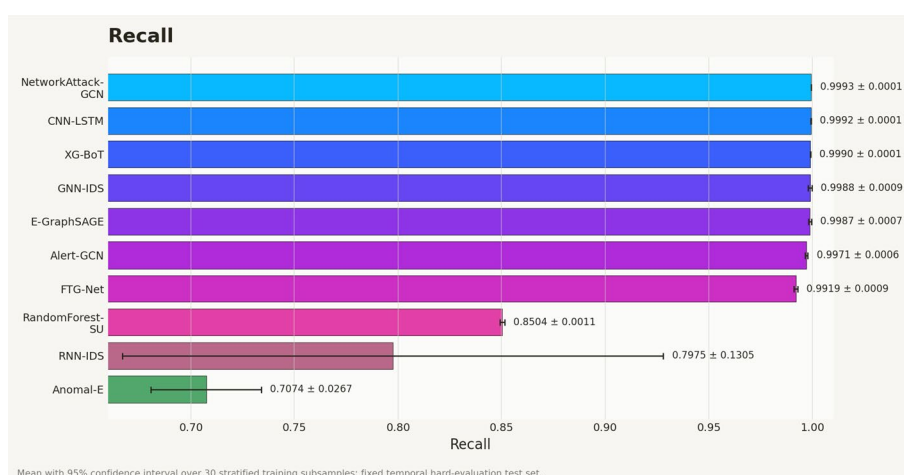


Fig. 16 AUC of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals; the y-axis is zoomed to make score-ranking differences visible

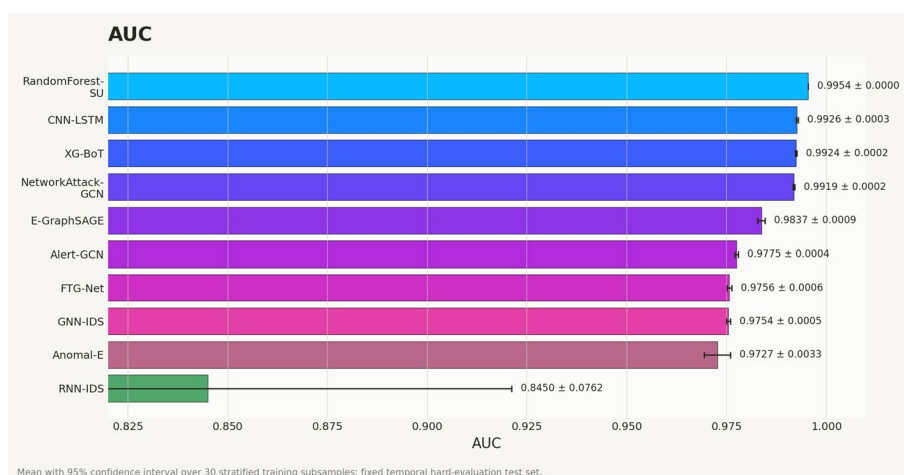


Fig. 17 Training time of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals

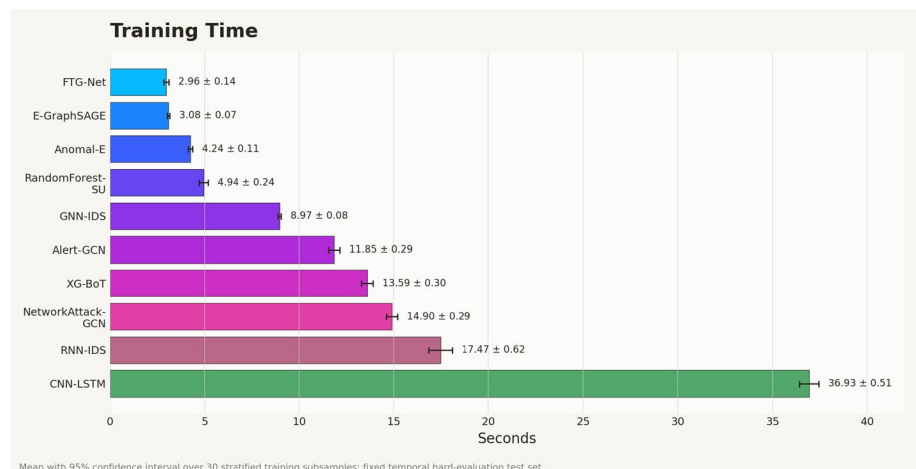
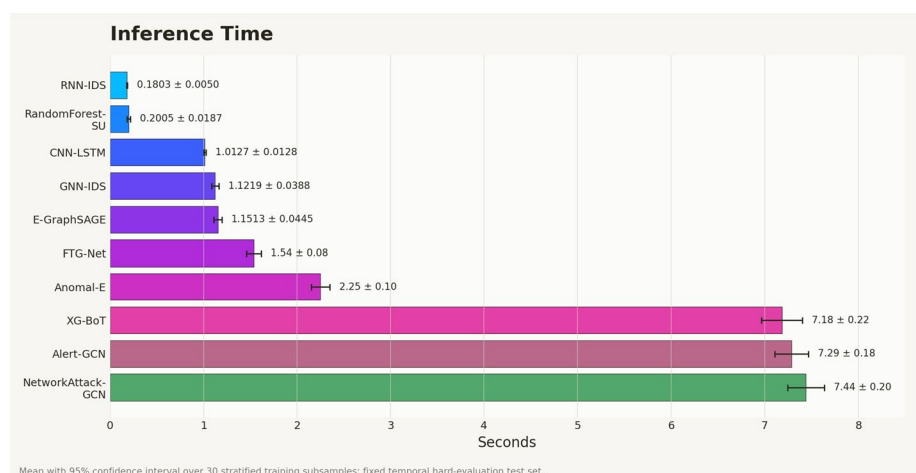


Fig. 18 Inference time of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals



reaches 0.9837, which is lower than the strongest score-ranking models but still high enough to support threshold-based deployment. The key interpretation is that RandomForest-SU is the best ranking model, while several graph-based models provide a better attack-coverage and timing profile.

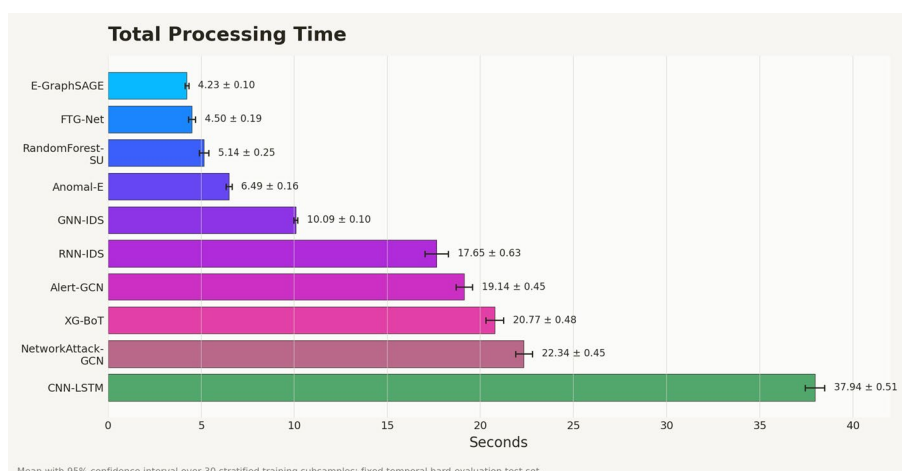
5.5 Training, inference, and total processing time

Timing results are central to real-world IDS feasibility. Figure 17 shows that FTG-Net has the shortest mean training time at 2.77 s, followed closely by E-GraphSAGE at 3.08 s. RandomForest-SU trains in 4.94 s, while CNN-LSTM requires 32.67 s. These results support the claim that efficient graph-based models can be retrained frequently without imposing the highest computational cost.

Inference time alone favors RNN-IDS and RandomForest-SU, as shown in Fig. 18. However, inference latency should be interpreted together with Detection Rate and total processing time. The fastest detector is not necessarily the most useful detector if it misses a larger fraction of attacks.

Total time provides the most compact operational summary. Figure 19 shows that E-GraphSAGE achieves the best mean total time at 4.23 s, followed by FTG-Net at 4.50 s and RandomForest-SU at 5.14 s. At the same time, E-GraphSAGE maintains a mean Detection Rate of 0.9987, compared with 0.8504 for RandomForest-SU. This combination makes E-GraphSAGE an attractive practical compromise: it is not the most precise model, but it offers near-perfect attack coverage with the best total processing time.

Fig. 19 Total processing time of the ten evaluated IDS models over 30 repeated training subsamples. Error bars show 95% confidence intervals



5.6 Operational interpretation

The results support a metric-dependent interpretation. RandomForest-SU is the best model for Accuracy, Precision, and AUC, and CNN-LSTM is highly competitive for F1-Score and Detection Rate. This outcome is not surprising on UNSW-NB15: many attacks are separable through engineered tabular flow statistics, so a strong tree ensemble can exploit feature thresholds and interactions without paying graph-construction or message-passing overhead. In such a setting, flat-feature sufficiency can outweigh the representational benefit of graph structure. However, the practical objective of operational IDS is not only to maximize aggregate correctness. A detector must also minimize missed attacks, support repeated retraining, and remain feasible under near-real-time constraints. On these criteria, efficient GNN-based models are more compelling. E-GraphSAGE is the strongest overall operational candidate because it combines the best total processing time with a near-perfect Detection Rate. FTG-Net is also attractive from a runtime perspective, while NetworkAttack-GCN and XG-BoT provide very strong Detection Rate and AUC at the cost of higher total time.

These findings should be interpreted as controlled benchmark evidence, not as a universal ranking across all IDS settings. UNSW-NB15 is a public benchmark with known limitations, and production deployment requires additional testing on streaming traffic, encrypted traffic, adversarially adapted attacks, graph-construction overhead, memory pressure, and data-plane constraints. This is why graph-based IDS evaluation should report not only classifier metrics, but also graph construction time, edge-list size, window duration, update frequency, inference latency, and memory footprint.

6 Conclusions and future works

This section synthesizes the main findings of the survey and quantitative evaluation. It summarizes the methodological lessons from the taxonomy, discusses the practical implications of the UNSW-NB15 case study, identifies remaining limitations in current GNN-IDS research, and outlines future directions for more robust, deployable, and statistically rigorous graph-based intrusion detection systems.

6.1 Key findings

The quantitative analysis supports a nuanced conclusion rather than a blanket claim that GNN-based IDS always outperforms traditional methods. The representative random-forest-based IDS baseline is the strongest conventional model family for aggregate tabular metrics in the UNSW-NB15 hard evaluation, reaching the highest mean

values for Accuracy, Precision, and AUC. The confidence intervals also show that some high-performing models are closely grouped on raw predictive metrics, which confirms that small decimal-level differences should not be over-interpreted. Traditional machine-learning models are therefore highly relevant when attacks are separable through flow-level features and when alert precision is a central requirement.

The GNN-based models, however, provide a stronger deployment-oriented profile. Enhanced GraphSAGE-style and GCN-based graph detectors detect almost all attacks in the hard evaluation slice, while alert-graph, generic GNN-IDS, and explainable graph-based botnet-detection representatives also achieve high Detection Rate and Recall. Several graph-based representatives also show better training or total-time efficiency than heavier deep sequential baselines. Thus, the main practical advantage of GNN-based IDS is not uniform superiority on every metric, but a favorable combination of attack coverage, scalability, low missed-attack burden, and near-real-time feasibility.

The literature synthesis and controlled evaluation indicate that graph representation choices substantially affect detection behavior. Dynamic, spatio-temporal, heterogeneous, and edge-aware representations are important because they can model evolving footprints of advanced persistent threats, botnets, lateral movement, and data exfiltration campaigns more naturally than static flow tables. The empirical case study should therefore be read as an illustrative controlled comparison, while broader claims about representation superiority require cross-dataset, repeated-run, and statistically validated benchmarking.

6.2 Architectural insights and performance analysis

The comparative analysis of GNN architectures reveals distinct performance-complexity trade-offs. Attention-based and hybrid GNN architectures, including GATs and enhanced GraphSAGE/GCN variants, can provide strong detection performance when combined with careful feature engineering and appropriate task granularity. GraphSAGE variants are particularly attractive in scalability scenarios because neighborhood sampling limits the amount of context processed per node, while GCN-style architectures offer efficient baseline performance suitable for resource-constrained environments.

Edge-aware graph modeling is a critical design consideration because network-flow information often resides on graph edges. Architectures that explicitly incorporate edge features in message-passing mechanisms can improve flow-level intrusion detection over purely node-centric approaches. This observation is consistent with the broader taxonomy: node-level, edge-level, and graph-level prediction tasks should be matched to the way traffic evidence is represented in the graph.

6.3 Identified research gaps and limitations

Despite significant progress, several limitations continue to constrain GNN-based IDS research. *First*, graph construction is under-specified in many studies. Reported performance can change substantially depending on whether graphs are built from IP endpoints, flow similarity, temporal co-occurrence, provenance relations, protocol fields, or heterogeneous entity types. Leakage-safe graph construction is therefore a methodological requirement, not an implementation detail.

Second, dataset limitations affect fairness. Public IDS benchmarks differ in attack prevalence, duplicate records, temporal continuity, traffic-generation realism, feature availability, and labeling quality. A model that performs well on UNSW-NB15 may not generalize to IoT telemetry, provenance logs, encrypted traffic, or cross-organization settings. The quantitative case study in this paper should therefore be interpreted as an illustrative controlled benchmark, not as a universal ranking across all IDS environments.

Third, real-time deployment is insufficiently validated. Many papers report classifier metrics but omit packet-capture assumptions, flow aggregation windows, graph update cost, edge construction overhead, memory footprint, and latency budgets. Practical GNN-IDS research should report end-to-end timing, including data collection, preprocessing, graph construction, model inference, and alert generation.

Fourth, adversarial robustness is still an open problem. GNN-based IDS can be attacked through feature manipulation, edge perturbation, graph poisoning, or realistic traffic-shaping strategies. Standard benchmark evaluation can overestimate resilience under drift and practical adversarial conditions. This survey therefore treats adversarial robustness as a research gap rather than as an experimentally validated property of the evaluated models.

Finally, statistical rigor is limited across the literature. Future benchmark studies should include repeated runs, confidence intervals, seed sensitivity, paired statistical tests, and cross-dataset validation. Where cross-validation is used, the number of folds, split ratios, stratification policy, temporal ordering, and leakage controls must be stated explicitly.

6.4 Future research directions

Future GNN-IDS research should move beyond reporting higher benchmark accuracy and toward deployable, auditable, and transferable systems. Five directions are especially important.

First, graph construction should become adaptive and leakage-aware. Future models should learn or select graph views under explicit constraints, including endpoint-free construction, temporal holdout, edge-budget limits, and robustness to missing or noisy features.

Second, IDS evaluation should shift from static benchmark tables to streaming and testbed-based protocols. This includes reporting graph-building overhead, window duration, packet-to-flow aggregation delay, memory consumption, and inference latency under realistic traffic rates.

Third, future work should combine GNNs with non-neural graph analytics. Causal graph analysis, knowledge graphs, random-walk scoring, and rule-based provenance reasoning can provide interpretability and operational constraints that neural message passing alone does not provide.

Fourth, adversarial evaluation must become standard. GNN-IDS papers should evaluate feature perturbation, edge manipulation, graph poisoning, traffic morphing, and distribution drift rather than treating robustness as a qualitative claim.

Fifth, pre-trained and self-supervised GNNs should be studied as deployment tools. Practical spatio-temporal models such as PPT-GNN argue that unlabeled pre-training and few-second graph windows can reduce dependence on labeled target-network data and improve transfer to unseen networks (Van Langendonck et al. 2025).

6.5 Implications for practice and policy

The findings of this survey have important implications for cybersecurity practitioners and policy makers. Organizations considering GNN-based IDS deployment should prioritize solutions that demonstrate robust performance across diverse network environments and attack types, rather than optimizing for single-dataset point estimates. The empirical findings suggest that GNN approaches can be particularly attractive when high attack coverage, low missed-attack burden, and near-real-time processing are more important than the cleanest possible alert stream. Their additional complexity is most justified in high-security environments where relational context and multi-hop interaction patterns are operationally valuable.

The need for standardized evaluation frameworks and benchmark datasets specifically designed for graph-based intrusion detection is critical for advancing the field and enabling objective comparison of proposed approaches. Funding agencies and research institutions should prioritize the development of such standardized resources.

6.6 Concluding remarks

Graph Neural Networks represent an important methodological shift in intrusion detection because they model traffic, hosts, alerts, and provenance events as relational structures rather than isolated records. The comprehensive analysis of this survey demonstrates both the potential and the current limitations of GNN-based approaches.

While significant challenges persist, particularly in adversarial robustness, real-time scalability, graph-construction overhead, and standardized evaluation, graph-based modeling offers a compelling way to represent cybersecurity problems where relational context matters.

Overall, GNNs are best understood as an important extension of IDS methodology rather than a universal replacement for traditional methods. Their value is strongest when detection depends on relational context, temporal interaction patterns, heterogeneous entities, or multi-hop attack behavior. The UNSW-NB15 analysis supports this interpretation: traditional models are highly competitive in aggregate tabular metrics, while GNN-based methods show stronger attack coverage and more practical timing behavior under the leakage-controlled temporal setting.

The next stage of GNN-IDS research should therefore focus on methodological credibility and operational readiness. This includes leakage-safe graph construction, benchmark fairness, statistical validation, deployment-oriented timing, adversarial robustness, and clearer comparison against non-neural graph analytics and traditional ML baselines. If these issues are addressed, GNN-based IDS can move from promising benchmark models toward reliable components of real-world cybersecurity infrastructure.

Author contributions Mohammad Reza Ismaeli Taba(M.R.I.), Hadi Zare(H.Z.), Saeed Rakhshani(S.R.) Conceptualization: M.R.I. and H.Z. Methodology: M.R.I., H.Z. and S.R. Software: M.R.I. Validation: M.R.I. and H.Z. Formal analysis: M.R.I. and H.Z. Investigation: M.R.I. and H.Z. Resources: H.Z. Data curation: M.R.I. Writing: M.R.I., H.Z. and S.R. Visualization: M.R.I. and S.R. Supervision: H.Z. Project administration: H.Z.

Funding This research received no funding.

Data availability We used a data analysis through a standard dataset.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Abdullayeva F, Suleymanzade S (2024) Cyber security attack recognition on cloud computing networks based on graph convolutional neural network and graphsage models. Results Control Optim 15:100423. <https://doi.org/10.1016/j.rico.2024.100423>
- Abinesh R, Yogeshkumar VG, Sarabesh TJ, Nandhini S (2024) Deep graph convolution neural network based intrusion detection system towards early detection of malicious attacks. In: 2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC). IEEE, pp 549–554. <https://doi.org/10.1109/i-smac61858.2024.10714598>
- Ahanger AS, Khan SM, Masoodi F et al (2025) Advanced intrusion detection in internet of things using graph attention networks. Sci Rep 15(1):9831. <https://doi.org/10.1038/s41598-025-94624-8>
- Ahmed U, Sadiq MT (2026) Enhancing SDN intrusion detection via multi-hybrid deep learning fusion and explainable AI. Mathematics 14(9):1498. <https://doi.org/10.3390/math14091498>
- Ahmed U, Zheng J, Almogren A et al (2024) Hybrid bagging and boosting with SHAP based feature selection for enhanced predictive modeling in intrusion detection systems. Sci Rep 14:30532. <https://doi.org/10.1038/s41598-024-81151-1>

- Ahmed U, Zheng J, Khan S et al (2025) Consensus hybrid ensemble machine learning for intrusion detection with explainable AI. *J Netw Comput Appl* 235:104091. <https://doi.org/10.1016/j.jnca.2024.104091>
- Akinyelu AA (2019) Machine learning and nature inspired based phishing detection: a literature survey. *Int J Artif Intell Tools* 28(05):1930002. <https://doi.org/10.1142/s0218213019300023>
- Alizadeh F, Khansari M (2023) An analysis of botnet detection using graph neural network. In: 2023 13th International Conference on Computer and Knowledge Engineering (ICCKE)
- Alrzini J, Pennington D (2020) A review of polymorphic malware detection techniques. *Int J Adv Res Eng Technol* 11(12):1238–1247
- Altat T, Wang X, Ni W et al (2023) NE-GConv: a lightweight node edge graph convolutional network for intrusion detection. *Comput Secur* 130:103285. <https://doi.org/10.1016/j.cose.2023.103285>
- Anoop M, Mary LW, Wilson AJ et al (2024) Optimized graph transformer with molecule attention network based multi class attack detection framework for enhancing privacy and security in WSN. *Multimed Tools Appl* 84(15):14273–14304. <https://doi.org/10.1007/s11042-024-19516-x>
- Bajpai S, Murthy PK, Kumar N (2024) Anomgraphadv: enhancing anomaly and network intrusion detection in wireless networks using adversarial training and temporal graph networks. In: *Proceedings of the 17th ACM Conference on Security and Privacy in Wireless and Mobile Networks*
- Bao H, Li W, Wang X, et al (2023) Payload level graph attention network for web attack traffic detection. In: *Lecture Notes in Computer Science*. Springer Nature Switzerland, pp 394–407. https://doi.org/10.1007/978-3-031-36030-5_32
- Barradas D, Santos N, Rodrigues L, et al (2021) Flowlens: enabling efficient flow classification for ML-based network security applications. In: *Proceedings of the network and distributed system security symposium*. Internet Society. <https://doi.org/10.14722/ndss.2021.24067>, <https://www.ndss-symposium.org/ndss-paper/flowlens-enabling-efficient-flow-classification-for-ml-based-network-security-applications/>
- Barsellotti L, De Marinis L, Cugini F, et al (2023) FTG-Net: hierarchical flow-to-traffic graph neural network for ddos attack detection. In: 2023 IEEE 24th international conference on high performance switching and routing (HPSR). IEEE, pp 173–178, <https://doi.org/10.1109/hpsr57248.2023.10147929>
- Bhumika, Das D (2022) Anom-dgcnn: Detection of anomalies in dynamic networks using deviated graph convolution network. In: 2022 International Wireless Communications and Mobile Computing (IWCMC). IEEE, pp 1273–1278, <https://doi.org/10.1109/iwcmc55113.2022.9824978>,
- Bibi I, Ozcelebi T, Meratnia N (2023) An iot attack detection framework leveraging graph neural networks. In: *International conference on intelligence of things*
- Bilot T, Madhoun NE, Agha KA, et al (2023a) A benchmark of graph augmentations for contrastive learning-based network attack detection with graph neural networks. In: 2023 7th Cyber Security in Networking Conference (CSNet). IEEE, pp 53–56, <https://doi.org/10.1109/csnet59123.2023.10339697>
- Bilot T, Madhoun NE, Agha KA et al (2023b) Graph neural networks for intrusion detection: a survey. *IEEE Access* 11:49114–49139. <https://doi.org/10.1109/access.2023.3275789>
- Bilot T, El Madhoun N, Al Agha K, et al (2024) Few edges are enough: few-shot network attack detection with graph neural networks. In: *Lecture notes in computer science*. Springer Nature Singapore, pp 257–276, https://doi.org/10.1007/978-981-97-7737-2_15
- Biswas P, Samanta T, Sanyal J (2023) Intrusion detection using graph neural network and lyapunov optimization in wireless sensor network. *Multimed Tools Appl* 82(9):14123–14134. <https://doi.org/10.1007/s11042-022-13992-9>
- Bora N, Chauhan A (2023) Edge-graph convolution network: an intrusion detection approach for industrial IoT. In: *International conference on data analytics & management*
- Boukari IE, Derdouha IA, Bouzeffrane S, et al (2024) StrucTemp-GNN: an intrusion detection framework in iot networks using dynamic heterogeneous graph neural networks. In: *Lecture Notes in Computer Science*. Springer, Switzerland, pp 17–39. https://doi.org/10.1007/978-3-031-52426-4_2
- Boyaci O, Narimani MR, Davis KR et al (2022) Joint detection and localization of stealth false data injection attacks in smart grids using graph neural networks. *IEEE Trans Smart Grid* 13(1):807–819. <https://doi.org/10.1109/tsg.2021.3117977>
- Buterez D, Janet JP, Kiddle SJ et al (2024) Transfer learning with graph neural networks for improved molecular property prediction in the multi-fidelity setting. *Nat Commun* 15(1):1517. <https://doi.org/10.1038/s41467-024-45566-8>
- Cai Q, Wang L, Zhu Y, et al (2026) Building next-generation datasets for provenance-based intrusion detection. In: *Workshop on attack provenance, reasoning, and investigation for security in the monitored environment (PRISM), co-located with NDSS*. NDSS Symposium. <https://www.ndss-symposium.org/ndss-paper/auto-draft-665/>
- Casas P, Vanerio J, Ullrich J, et al (2023) Graphsec – advancing the application of AI/ML to network security through graph neural networks. In: *Lecture Notes in Computer Science*. Springer, Switzerland, pp 56–71. https://doi.org/10.1007/978-3-031-36183-8_5
- Caville E, Lo WW, Layeghy S et al (2022) Anomal-E: a self-supervised network intrusion detection system based on graph neural networks. *Knowl-Based Syst* 258:110030. <https://doi.org/10.1016/j.knosys.2022.110030>
- Chang L, Branco P (2024) Embedding residuals in graph-based solutions: the E-ResSAGE and E-ResGAT algorithms. A case study in intrusion detection. *Appl Intell* 54(8):6025–6040. <https://doi.org/10.1007/s10489-024-05404-2>

- Che X, Li H, Chi L, et al (2023) Evaluation the robustness of IoT intrusion detection models in adversarial attack environments. <https://doi.org/10.2139/ssrn.4611920>
- Chen B, Wu QH, Li M et al (2023a) Detection of false data injection attacks on power systems using graph edge-conditioned convolutional networks. *Prot Control Mod Power Syst* 8(1):1–12. <https://doi.org/10.1186/s41601-023-00287-w>
- Chen S, Lin D, Xie Z, et al (2023b) Temporal-gated graph neural network with graph sampling for multi-step attack detection. In: 2023 IEEE 22nd international conference on trust, security and privacy in computing and communications (TrustCom). IEEE, pp 1266–1271. <https://doi.org/10.1109/trustcom60117.2023.00172>
- Chen J, Xie H, Cai S et al (2024) GCN-MHSA: a novel malicious traffic detection method based on graph convolutional neural network and multi-head self-attention mechanism. *Comput Secur* 147:104083. <https://doi.org/10.1016/j.cose.2024.104083>
- Cheng Q, Wu C, Zhou S (2021) Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. *IEEE Commun Lett* 25(5):1564–1567. <https://doi.org/10.1109/lcomm.2020.3048995>
- Cheng S, Jin D, Ma Y et al (2024) LinkGuard: link flooding attack detection and mitigation via spatio-temporal graph convolutional network. *IEEE Trans Netw Sci Eng* 11(5):4059–4075. <https://doi.org/10.1109/tNSE.2024.3393513>
- Cuong NH, Hua ZL, do Xuan C (2023) Using inference and graph convolutional networks for apt attack detection. In: 2023 RIVF International conference on computing and communication technologies (RIVF)
- DARPA (2018) Transparent computing (TC) program datasets. DARPA cyber hunting at scale (CHASE) program. <https://github.com/darpa-i2o/Transparent-Computing>, includes Theia, TRACE, CADETS, FiveDirections sub-datasets
- Das S, Ghosh A, Chatterjee D (2024) Detection of false data injection attacks in cyber-physical systems. In: 2024 IEEE international symposium on information theory (ISIT)
- Deldar F, Abadi M (2024) Deep learning for zero-day malware detection and classification: a survey. *ACM Comput Surv* 56(2):1–37. <https://doi.org/10.1145/3605775>
- Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res* 7:1–30
- Deng P, Huang Y (2025) Edge-featured multi-hop attention graph neural network for intrusion detection system. *Comput Secur* 148:104132. <https://doi.org/10.1016/j.cose.2024.104132>
- Ding H, Chen L, Li S, et al (2024) Divide, conquer, and coalesce: meta parallel graph neural network for iot intrusion detection at scale. In: Proceedings of the ACM web conference 2024. ACM, pp 1656–1667. <https://doi.org/10.1145/3589334.3645457>
- Du P, Peng C, Xiang P, et al (2022) Anomaly detection of traffic session based on graph neural network. In: Proceedings of the 2022 international conference on cyber security. ACM, pp 1–9. <https://doi.org/10.1145/3584714.3584715>
- El-Dawy S (2024) Safeguarding smart vehicles: GNN-powered real-time IDs for can networks. In: 2024 international conference on machine intelligence and smart innovation (ICMISI)
- Ennaji S, de Gaspari F, Hitaj D et al (2025) Adversarial challenges in network intrusion detection systems: research insights and future prospects. *IEEE Access* 13:148613–148645. <https://doi.org/10.1109/access.2025.3600984>
- Fang Y, Huang C, Zeng M et al (2022) JStrong: malicious javascript detection based on code semantic representation and graph neural network. *Comput Secur* 118:102715. <https://doi.org/10.1016/j.cose.2022.102715>
- Farrukh YA, Wali S, Khan I et al (2025) XG-NID: dual-modality network intrusion detection using a heterogeneous graph neural network and large language model. *Expert Syst Appl* 287:128089. <https://doi.org/10.1016/j.eswa.2025.128089>
- Feng P, Wei D, Li Q et al (2024) Glareshell: graph learning-based PHP webshell detection for web server of industrial internet. *Comput Netw* 245:110406. <https://doi.org/10.2139/ssrn.4725152>
- Ferretti S, D'angelo G, Ghini V (2024) On the use of heterogeneous graph neural networks for detecting malicious activities: a case study with cryptocurrencies. In: Proceedings of the 4th international workshop on open challenges in online social networks
- Five Directions (2020) DARPA operationally transparent cyber (OpTC) dataset. DARPA cyber hunting at scale (CHASE) program. <https://github.com/FiveDirections/OpTC-data>
- Frenken R, Bhatti SG, Zhang H, et al (2025) KD-GAT: combining knowledge distillation and graph attention transformer for a controller area network intrusion detection system. In: 2025 IEEE 28th international conference on intelligent transportation systems (ITSC). IEEE, pp 3721–3726. <https://doi.org/10.1109/itsc60802.2025.11423252>
- Friji H, Mavromatis I, Sanchez-Mompo A, et al (2023) Multi-stage attack detection and prediction using graph neural networks: an IoT feasibility study. In: 2023 IEEE 22nd international conference on trust, security and privacy in computing and communications (TrustCom). IEEE, pp 620–627. <https://doi.org/10.1109/trustcom60117.2023.00095>
- Fu C, Li Q, Shen M et al (2023) Frequency domain feature based robust malicious traffic detection. *IEEE/ACM Trans Netw* 31(1):452–467. <https://doi.org/10.1109/TNET.2022.3195871>
- Fu C, Li Q, Shen M, et al (2024a) Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns. In: Proceedings of the 2024 ACM SIGSAC conference on computer and communications security. ACM, pp 3659–3673. <https://doi.org/10.1145/3658644.3670353>
- Fu C, Li Q, Xu K (2024b) Flow interaction graph analysis: unknown encrypted malicious traffic detection. *IEEE/ACM Trans Netw* 32(4):2972–2987. <https://doi.org/10.1109/TNET.2024.3370851>

- Fu C, Li Q, Bertino E, et al (2025) Training with only 1.0‰ samples: malicious traffic detection via cross-modality feature fusion. In: Proceedings of the 2025 ACM SIGSAC conference on computer and communications security. ACM, pp 3930–3944. <https://doi.org/10.1145/3719027.3765143>
- Galli D, Venturi A, Stabili D, et al (2025) Defending network intrusion detection systems based on graph neural networks against structural adversarial attacks. In: 2025 23rd international symposium on network computing and applications (NCA). IEEE, pp 219–228. <https://doi.org/10.1109/nca67271.2025.00043>
- Gao L, Fu C, Deng X, et al (2025) Wedjat: detecting sophisticated evasion attacks via real-time causal analysis. In: Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining. ACM, pp 342–353. <https://doi.org/10.1145/3690624.3709218>
- Gharavian V, Khosrowshahli R, Mahmoud QH, et al (2023) Intrusion detection for wireless sensor network using graph neural networks. In: 2023 IEEE symposium series on computational intelligence (SSCI). IEEE, pp 807–813. <https://doi.org/10.1109/ssci52147.2023.10372004>
- Gorricho-Segura M, Echeberria-Barrio X, Seguro-la-gil L (2023) Edge-based analysis for network intrusion detection using a GNN approach
- Grover A, Leskovec J (2016) Node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining
- Gu Z, Lopez DT, Alrahis L, et al (2024) Always be pre-training: representation learning for network intrusion detection with GNNs. In: 2024 25th international symposium on quality electronic design (ISQED). IEEE, pp 1–8. <https://doi.org/10.1109/isqed60706.2024.10528371>
- Guo Q, Li J (2024) A novel adaptive model based on graph neural network for cross-domain APT detection in industrial systems. In: Proceedings of the 2024 16th international conference on computer modeling and simulation. ACM, pp 48–54. <https://doi.org/10.1145/3686812.3686820>
- Ha TN, Romero D (2024) Spoofing detection in the physical layer with graph neural networks. In: 2024 IEEE 99th vehicular technology conference (VTC2024-Spring)
- Han Y, Feng H, Li K et al (2023) False data injection attacks detection with modified temporal multi-graph convolutional network in smart grids. *Comput Secur* 124:103016. <https://doi.org/10.1016/j.cose.2022.103016>
- Han X, Zhang M, Yang Z (2024) TBA-GNN: a traffic behavior analysis model with graph neural networks for malicious traffic detection. In: International conference on wireless artificial intelligent computing systems and applications
- Han X, Xu G, Zhang M et al (2024) DE-GNN: dual embedding with graph neural network for fine-grained encrypted traffic classification. *Comput Netw* 245:110372. <https://doi.org/10.1016/j.comnet.2024.110372>
- Hassan W, Hosseini SE, Pervez S (2023) Real-time anomaly detection in network traffic using graph neural networks and random forest. In: International conference on next generation wired/wireless networking
- Highnam K, Arulkumaran K, Hanif Z, et al (2021) BETH dataset: real cybersecurity data for anomaly detection research. In: ICML 2021 workshop on uncertainty and robustness in deep learning. <https://www.kaggle.com/datasets/katehighnam/beth-dataset>
- Holland J, Schmitt P, Feamster N, et al (2021) New directions in automated traffic analysis. In: Proceedings of the 2021 ACM SIGSAC conference on computer and communications security. ACM, pp 3366–3383. <https://doi.org/10.1145/3460120.3484758>
- Huang Z, Gu Y, Zhao Q (2022) One-class directed heterogeneous graph neural network for intrusion detection. In: Proceedings of the 2022 6th international conference on innovation in artificial intelligence
- Huang Y, Negrete J, Wagener J et al (2023) Graph neural networks and cross-protocol analysis for detecting malicious IP addresses. *Complex Intell Syst* 9(4):3857–3869. <https://doi.org/10.1007/s40747-022-00838-y>
- Ji Y, Huang HH (2022) Nestedgnn: Detecting malicious network activity with nested graph neural networks. In: ICC 2022-IEEE international conference on communications
- Jianping W, Guangqiu Q, Chunming W et al (2024) Federated learning for network attack detection using attention-based graph neural networks. *Sci Rep* 14(1):19088. <https://doi.org/10.1038/s41598-024-70032-2>
- Kent AD (2015) Comprehensive, multi-source cyber-security events. Los Alamos National Laboratory. <https://doi.org/10.17021/1179829>, <https://csr.lanl.gov/data/cyber1/>
- Khraisat A, Gondal I, Vamplew P et al (2019) Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity* 2(1):20. <https://doi.org/10.1186/s42400-019-0038-7>
- King IJ, Shu X, Jang J, et al (2023) EdgeTorrent: real-time temporal graph representations for intrusion detection. In: Proceedings of the 26th international symposium on research in attacks, intrusions and defenses. ACM, pp 77–91. <https://doi.org/10.1145/3607199.3607201>
- Kisanga P (2023) Network anomaly detection using a graph neural network. In: 2023 international conference on computing, networking and communications (ICNC)
- Koroniotis N, Moustafa N, Sitnikova E et al (2019) Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. *Futur Gener Comput Syst* 100:779–796. <https://doi.org/10.1016/j.future.2019.05.041> (<https://research.unsw.edu.au/projects/bot-iot-dataset>)

- Kumar K, Khari M (2025) Graph neural network-based malicious node detection to improve the security of wireless sensor networks. In: *Lecture notes in networks and systems*. Springer, Singapore, pp 363–372. https://doi.org/10.1007/978-981-97-6106-7_22
- Lakha B, Mount SL, Serra E, et al (2022) Anomaly detection in cybersecurity events through graph neural network and transformer based model: a case study with beth dataset. In: *2022 IEEE international conference on big data (Big Data)*. IEEE, pp 5756–5764. <https://doi.org/10.1109/bigdata55660.2022.10020336>
- Latif H, Suárez-Varela J, Cabellos-Aparicio A, et al (2023) Detecting contextual network anomalies with graph neural networks. In: *Proceedings of the 2nd on graph neural networking workshop 2023*. ACM, pp 25–30. <https://doi.org/10.1145/3630049.3630171>
- Le H-D, Park M (2024) Enhanced graphsage for multi-class intrusion detection. In: *2024 International conference on information networking (ICOIN)*
- Li X (2023) Network intrusion detection with edge-directed graph multi-head attention networks. [arXiv:2310.17348](https://arxiv.org/abs/2310.17348)
- Li Y, Li R, Zhou Z, et al (2022) Graphddos: Effective DDOS attack detection using graph neural networks. In: *2022 IEEE 25th international conference on computer supported cooperative work in design (CSCWD)*. IEEE, pp 1275–1280. <https://doi.org/10.1109/cscwd54268.2022.9776097>
- Li J, Zhou J, Li L, et al (2023a) Robust detection for adversarial attacks—based on a graph neural network for web application security protection system. In: *Proceedings of the 4th international conference on artificial intelligence and computer engineering*. ACM, pp 927–932. <https://doi.org/10.1145/3652628.3652782>
- Li X, Wang Y, Lu Z (2023b) Graph-based detection for false data injection attacks in power grid. *Energy* 263:125865. <https://doi.org/10.1016/j.energy.2022.125865>
- Liu X, Zeng H, Chand N, et al (2015) Experimental demonstration of high-throughput low-latency mobile fronthaul supporting 48 20-MHz LTE signals with 59-GB/s CPRI-equivalent rate and 2- μ s processing latency. In: *2015 European conference on optical communication (ECOC)*. IEEE, pp 1–3. <https://doi.org/10.1109/ecoc.2015.7342015>
- Liu Z, Chen C, Yang X, et al (2018) Heterogeneous graph neural networks for malicious account detection. In: *Proceedings of the 27th ACM international conference on information and knowledge management*. ACM, pp 2077–2085. <https://doi.org/10.1145/3269206.3272010>
- Liu X, Ren J, He H et al (2021) A fast all-packets-based DDOS attack detection approach based on network graph and graph kernel. *J Netw Comput Appl* 185:103079. <https://doi.org/10.1016/j.jnca.2021.103079>
- Liu Z, Fang Y, Huang C et al (2022) GraphXSS: an efficient XSS payload detection approach based on graph convolutional network. *Comput Secur* 114:102597. <https://doi.org/10.1016/j.cose.2021.102597>
- Liu C, Zhao J, Song Z, et al (2023) Anomalous traffic detection method for power internet of things based on graph neural networks. In: *2023 7th international conference on electrical, mechanical and computer engineering (ICEMCE)*. IEEE, pp 83–86. <https://doi.org/10.1109/icemce60359.2023.10490660>
- Lo WW, Layeghy S, Sarhan M, et al (2022) E-graphSAGE: a graph neural network based intrusion detection system for IoT. In: *NOMS 2022-2022 IEEE/IFIP network operations and management symposium*. IEEE, pp 1–9. <https://doi.org/10.1109/noms54207.2022.9789878>
- Lo WW, Kulatilleke G, Sarhan M et al (2023) XG-BoT: an explainable deep graph neural network for botnet detection and forensics. *Internet Things* 22:100747. <https://doi.org/10.1016/j.iot.2023.100747>
- Long J, Luo C, Chen R, et al (2023) Adaboost-based graph convolutional networks with channel attention for intrusion detection in internet-of-things. In: *2023 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom)*. IEEE, pp 207–212. <https://doi.org/10.1109/ispa-bdcloud-socialcom-sustaincom59178.2023.00060>
- Lv Z, Wang M, Chang H, et al (2023) A power monitoring network attack detection mechanism based on graph convolutional neural network. In: *2023 international conference on networks, communications and intelligent computing (NCIC)*. IEEE, pp 110–115. <https://doi.org/10.1109/ncic61838.2023.00025>
- Lakshmanan M, Adnan MM, Reddy RA, et al (2024) A graph neural network and transformer encoder technique for anomaly and cyber threat detection in smart grids. In: *2024 International conference on intelligent algorithms for computational intelligence systems (IACIS)*. IEEE, pp 1–4. <https://doi.org/10.1109/iacis61494.2024.10721753>
- Maksimski A (2023) Bonet detection mechanism using graph neural network. In: *International conference on advanced information networking and applications*
- Manikandan A, Singh AK, Chauhan A (2024) GES intrusion detection approach based on graph neural network for industrial IoT. In: *International conference on computing and communication systems for industrial applications*
- Manjula HS, Roopa MS, Arunalatha JS, et al (2023) Intrusion detection model for IoT networks using graph convolution networks(gcn). In: *Smart innovation, systems and technologies*. Springer, Singapore, pp 1–12. https://doi.org/10.1007/978-981-99-3982-4_1
- Marfo W, Tosh DK, Moore SV (2024) Enhancing network anomaly detection using graph neural networks. In: *2024 22nd Mediterranean Communication and Computer Networking Conference (MedComNet)*
- Messai M-L, Seba H (2023) IoT network attack detection: leveraging graph learning for enhanced security. In: *Proceedings of the 18th International conference on availability, reliability and security*

- Mirlashari M, Rizvi SAM (2024) Enhancing IoT intrusion detection system with modified E-graphSAGE: a graph neural network approach. *Int J Inf Technol* 16(4):2705–2713. <https://doi.org/10.1007/s41870-024-01746-9>
- Mohammadian H, Higgins G, Ansong S, et al (2025) Explainable malware detection through integrated graph reduction and learning techniques. *Big Data Research* <https://www.unb.ca/cic/datasets/sgg-dataset-2024.html>, related paper for the CIC-SGG-2024 dataset
- Mora-Gimeno FJ, Mora-Mora H, Volckaert B, Atrey A (2021) Intrusion detection system based on integrated system calls graph and neural networks. *IEEE Access* 9:9822–9833
- Moustafa N (2021) A new distributed architecture for evaluating AI-based security systems at the edge: network TON_IoT datasets. *Sustain Cities Soc* 72:102994. <https://doi.org/10.1016/j.scs.2021.102994>
- Moustafa N, Slay J (2015) UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). IEEE, pp 1–6. <https://doi.org/10.1109/MilCIS.2015.7348942>
- Moustafa N, Slay J (2016) The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Inf Secur J A Glob Perspect* 25(1–3):18–31. <https://doi.org/10.1080/19393555.2015.1125974>
- Nascita A, Aceto G, Ciuonzo D et al (2025) A survey on explainable artificial intelligence for internet traffic classification and prediction, and intrusion detection. *IEEE Commun Surv Tutor* 27:3165–3198. <https://doi.org/10.1109/COMST.2024.3504955>
- Neto ECP, Dadkhah S, Ferreira R et al (2023) CIIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* 23(13):5941. <https://doi.org/10.3390/s23135941>
- Okui N, Akimoto Y, Kubota A, et al (2023) A graph construction method for anomalous traffic detection with graph neural networks using sets of flow data. In: 2023 IEEE 47th annual computers, software, and applications conference (COMP-SAC). IEEE, pp 1017–1018. <https://doi.org/10.1109/compsac57700.2023.00154>
- Pan H, Fang Y, Guo W et al (2024) Few-shot graph classification on cross-site scripting attacks detection. *Comput Secur* 140:103749. <https://doi.org/10.1016/j.cose.2024.103749>
- Park SB, Jo HJ, Lee DH (2023) G-IDCS: graph-based intrusion detection and classification system for can protocol. *IEEE Access* 11:39213–39227. <https://doi.org/10.1109/access.2023.3268519>
- Peng C, Zhang Y (2024) Industrial internet of things intrusion detection model integrating graph attention network and gated temporal convolutional network. In: 2024 7th international conference on advanced algorithms and control engineering (ICAACE). IEEE, pp 596–599. <https://doi.org/10.1109/icaace61206.2024.10549297>
- Pramanick N, Mathew J, Selvarajan S et al (2025) Leveraging stacking machine learning models and optimization for improved cyberattack detection. *Sci Rep* 15(1):16757. <https://doi.org/10.1038/s41598-025-01052-9>
- Pujol-Perich D, Suárez-Varela J, Cabellos-Aparicio A, Barlet-Ros P (2022) Unveiling the potential of graph neural networks for robust intrusion detection. *ACM Sigmetrics Perform Evaluation Rev* 49(4):111–117
- Purnama SR, Istiyanto JE, Amrizal MA, et al (2022) Inductive graph neural network with causal sampling for IoT network intrusion detection system. In: 2022 International conference on computer engineering, network, and intelligent multimedia (CENIM). IEEE, pp 241–246. <https://doi.org/10.1109/cenim56801.2022.10037304>
- Reka R, Karthick R, Saravana Ram R et al (2024) Multi head self-attention gated graph convolutional network based multi-attack intrusion detection in manet. *Comput Secur* 136:103526. <https://doi.org/10.1016/j.cose.2023.103526>
- Ren J, Geng R (2025) Provenance-based apt campaigns detection via masked graph representation learning. *Comput Secur* 148:104159. <https://doi.org/10.1016/j.cose.2024.104159>
- Ren W, Song X, Hong Y et al (2023) Apt attack detection based on graph convolutional neural networks. *Int J Comput Intell Syst* 16(1):184. <https://doi.org/10.1007/s44196-023-00369-5>
- Roy KC, Chen Q (2024) Logshield: A graph-based real-time anomaly detection framework using frequency analysis. In: International conference on science of cyber security
- Ruiz L (2023a) L.F.O.C., Alejandro Ribeiro, graphon neural networks and the transferability of graph neural networks
- Ruiz L (2023b) Transferability properties of graph neural networks_supp1-3297848.pdf. *IEEE Trans Signal Process* 71:3474–3489. <https://doi.org/10.1109/tsp.2023.3297848/mm1>
- Sabir B, Ullah F, Babar MA et al (2022) Machine learning for detecting data exfiltration. *ACM Comput Surv* 54(3):1–47. <https://doi.org/10.1145/3442181>
- Sadrolhefazi ZS, Zare H, Asghari SA, et al (2024) Unsupervised graph-based anomaly detection using variational autoencoder. In: 2024 IEEE International conference on data mining workshops (ICDMW). IEEE, pp 734–741. <https://doi.org/10.1109/icdmw65004.2024.00101>
- Sarhan M, Layeghy S, Moustafa N, et al (2021) NetFlow datasets for machine learning-based network intrusion detection systems. In: Big data technologies and applications (BDTA 2020 / WiCON 2020). Springer, Cham. pp 117–135. https://doi.org/10.1007/978-3-030-72802-1_9, https://staff.itee.uq.edu.au/marius/NIDS_datasets/
- Saunders BJ, Kisanga P, Carvalho GH, et al (2024) A graph convolutional networks-based ddos detection model. In: 2024 IEEE international systems conference (SysCon). IEEE, pp 1–5. <https://doi.org/10.1109/syscon61195.2024.10553611>

- Sharafaldin I, Lashkari AH, Ghorbani AA (2018) Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th international conference on information systems security and privacy (ICISSP). SCITEPRESS, Madeira, Portugal, pp 108–116. <https://doi.org/10.5220/0006639801080116>, <https://www.unb.ca/cic/datasets/ids-2017.html>
- Shareef SK, Chaitanya RK, Chennupalli S et al (2024) Enhanced botnet detection in IoT networks using zebra optimization and dual-channel GAN classification. *Sci Rep* 14(1):17148. <https://doi.org/10.1038/s41598-024-67865-2>
- Sharma A, et al. (2022) Graph neural network-based anomaly detection in blockchain network. In: International conference on computing, communications, and cyber-security
- Shi H, Ji L, Liu S, et al (2021) Multi-layer graph neural network-based random anomalous behavior detection. In: 2021 International conference on digital society and intelligent systems (DSInS). IEEE, pp 266–270. <https://doi.org/10.1109/dsins54396.2021.9670589>
- Shokouhinejad H, Higgins G, Razavi-Far R, et al (2025) On the consistency of GNN explanations for malware detection. *Information Sciences* <https://www.unb.ca/cic/datasets/cic-dgg-2025.html>, related paper for the CIC-DGG-2025 dataset
- Singh SP, Abhay B, Kumar A, et al (2024) An efficient approach using graphical neural network for network intrusion detection system. In: 2024 15th International conference on computing communication and networking technologies (ICCCNT). IEEE, pp 1–11. <https://doi.org/10.1109/iccant61001.2024.10725313>
- Skarding J, Gabrys B, Musial K (2021) Foundations and modeling of dynamic networks using dynamic graph neural networks: a survey. *IEEE Access* 9:79143–79168. <https://doi.org/10.1109/access.2021.3082932>
- Song J, Qin G, Liang Y et al (2024) DGIDS: dynamic graph-based intrusion detection system for can. *Comput Secur* 147:104076. <https://doi.org/10.1016/j.cose.2024.104076>
- Sudhakar KS (2020) An emerging threat fileless malware: a survey and research challenges. *Cybersecurity* 3(1):1. <https://doi.org/10.1186/s42400-019-0043-x>
- Sun Z, Teixeira AM, Toor S (2024) GNN-IDS: graph neural network based intrusion detection system. In: Proceedings of the 19th international conference on availability, reliability and security
- Takiddin A, Atat R, Ismail M et al (2023a) Generalized graph neural network-based detection of false data injection attacks in smart grids. *IEEE Trans Emerg Top Comput Intell* 7(3):618–630. <https://doi.org/10.1109/tetci.2022.3232821>
- Takiddin A, Atat R, Ismail M, et al (2023b) A graph neural network multi-task learning-based approach for detection and localization of cyberattacks in smart grids. In: ICASSP 2023 - 2023 IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, pp 1–5. <https://doi.org/10.1109/icassp49357.2023.10096822>
- Tang Q, Chen H, Ge B et al (2022) AIGCN: attack intention detection for power system using graph convolutional networks. *J Signal Process Syst* 94(11):1119–1127. <https://doi.org/10.1007/s11265-021-01724-5>
- Tang M, Tang W, Gui Q et al (2024) A vulnerability detection algorithm based on residual graph attention networks for source code imbalance (RGAN). *Expert Syst Appl* 238:122216. <https://doi.org/10.1016/j.eswa.2023.122216>
- Tankard C (2011) Advanced persistent threats and how to monitor and deter them. *Netw Secur* 2011(8):16–19. [https://doi.org/10.1016/s1353-4858\(11\)70086-1](https://doi.org/10.1016/s1353-4858(11)70086-1)
- Tian Z, Zhang D, Dai HN (2024) Continual learning on graphs: a survey. *arXiv preprint* <https://doi.org/10.48550/arXiv.2402.06330>, <https://arxiv.org/abs/2402.06330>
- Tian Q, Zhao C, Shao M, et al (2025) Mldgg: Meta-learning for domain generalization on graphs. In: Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining V.1. ACM, pp 1361–1372. <https://doi.org/10.1145/3690624.3709188>
- Ullah S, Boulila W, Koubâa A et al (2023) MAGRU-IDS: a multi-head attention-based gated recurrent unit for intrusion detection in IIoT networks. *IEEE Access* 11:114590–114601. <https://doi.org/10.1109/access.2023.3324657>
- Van Langendonck L, Castell-Uroz I, Barlet-Ros P (2025) PPT-GNN: a practical pretrained spatio-temporal graph neural network for network security. In: 2025 IEEE European symposium on security and privacy workshops (EuroS&PW). IEEE, pp 169–175. <https://doi.org/10.1109/eurospw67616.2025.00026>
- Venturi A, Ferrari M, Marchetti M, et al (2023) Arganids: a novel network intrusion detection system based on adversarially regularized graph autoencoder. In: Proceedings of the 38th ACM/SIGAPP symposium on applied computing. ACM, pp 1540–1548. <https://doi.org/10.1145/3555776.3577651>
- Wang X (2023) Identification YZNA, on graph convolutional neural network. In 2023 IEEE 22nd international conference on trust AB (2023b) Security and privacy in computing and communications (trustcom)
- Wang Y, Jiang Y, Lan J (2021) Intrusion detection using few-shot learning based on triplet graph convolutional network. *J Web Eng* 20(5):1527–1552. <https://doi.org/10.13052/jwe1540-9589.2059>
- Wang M, Li P, Cheng Z, et al (2023a) Unsupervised graph-sequence anomaly detection for 5G core network control plane traffic. In: 2023 IEEE 29th international conference on parallel and distributed systems (ICPADS). IEEE, pp 1645–1652. <https://doi.org/10.1109/icpads60453.2023.00230>
- Wang Y, Han Z, Li J, He X (2023b) BS-GAT behavior similarity based graph attention network for network intrusion detection. *arXiv:2304.07226*
- Wang C, Tan G, Roy SB, et al (2024) Distribution-aware online continual learning for urban spatio-temporal forecasting. *arXiv preprint* [arXiv:2411.15893](https://doi.org/10.48550/arXiv.2411.15893), <https://doi.org/10.48550/arXiv.2411.15893>, <https://arxiv.org/abs/2411.15893>

- Wei C, Xie G, Diao Z (2023a) Network flow based iot anomaly detection using graph neural network. In: International conference on knowledge science, engineering and management
- Wei X, Liu W, Zhou J, et al (2023b) Detecting false data injection attacks using spatial-temporal graph neural network. In: 2023 IEEE 12th data driven control and learning systems conference (DDCLS). IEEE, pp 338–343. <https://doi.org/10.1109/ddcls58216.2023.10166952>
- Wu Y, Nie L, Xiong X et al (2024) Incremental update intrusion detection for industry 5.0 security: a graph attention network-enabled approach. *IEEE Trans Consum Electron* 70(1):2004–2017. <https://doi.org/10.1109/tce.2023.3331907>
- Xiao J, Yang L, Zhong F et al (2023) Robust anomaly-based insider threat detection using graph neural network. *IEEE Trans Netw Serv Manage* 20(3):3717–3733. <https://doi.org/10.1109/tnsm.2022.3222635>
- Xie S, Zhan C, Li J, et al (2025) Intrusion detection method based on graph edge attention and focal loss. In: Proceedings of the 2025 4th international conference on cryptography, network security and communication technology. ACM, pp 21–28. <https://doi.org/10.1145/3723890.3723895>
- Xu S, Qian Y, Hu RQ (2023a) Adversarial examples: challenges and solutions. *Cybersecurity Intell Netw Syst*. <https://doi.org/10.1002/9781119784135.ch8>
- Xu Y, Fang Y, Liu Z et al (2023b) PWAGAT: potential web attacker detection based on graph attention network. *Neurocomputing* 557:126725. <https://doi.org/10.1016/j.neucom.2023.126725>
- Xu B, Gong Y, Geng X et al (2024a) ProcSAGE: an efficient host threat detection method based on graph representation learning. *Cybersecurity* 7(1):51. <https://doi.org/10.1186/s42400-024-00240-w>
- Xu R, Wu G, Wang W et al (2024b) Applying self-supervised learning to network intrusion detection for network flows with graph neural network. *Comput Netw* 248:110495. <https://doi.org/10.1016/j.comnet.2024.110495>
- Xu J, Li C, Zheng Y (2026) Entente: Cross-silo intrusion detection on network log graphs with federated learning. Proceedings 2026 network and distributed system security symposium. Internet Society. <https://doi.org/10.14722/ndss.2026.230093>
- Yang S, Wu Q (2024) SSGCRTN: a space-specific graph convolutional recurrent transformer network for traffic prediction. *Appl Intell* 54(22):11978–11994. <https://doi.org/10.1007/s10489-024-05815-1>
- Yang S, Wu Q, Wang Y et al (2025) MSTDFGRN: a multi-view spatio-temporal dynamic fusion graph recurrent network for traffic flow prediction. *Comput Electr Eng* 123:110046. <https://doi.org/10.1016/j.compeleceng.2024.110046>
- You I, Yim K (2010) Malware obfuscation techniques: A brief survey. In: 2010 International conference on broadband, wireless computing, communication and applications. IEEE, pp 297–300. <https://doi.org/10.1109/BWCCA.2010.85>
- Zade HR (2024) H.Z., Mohsen Ghassemi Parsa, Hadi Davardoust & Meshkat Shariat Bagheri DCOR: anomaly detection in attributed networks via dual contrastive learning reconstruction
- Zambare P, Liu Y (2024) An optimized graph neural network-based approach for intrusion detection in smart vehicles. In: IFIP advances in information and communication technology. Springer, Switzerland, pp 3–17. https://doi.org/10.1007/978-3-031-45878-1_1
- Zhang X (2023) A survey of large-scale complex information network representation learning methods. In: 2023 3rd international conference on consumer electronics and computer engineering (ICCECE), vol 6. IEEE, pp 118–123. <https://doi.org/10.1109/iccece58074.2023.10135535>
- Zhang Y, Zhang Y, Wu Y, et al (2022) TPE-NIDS: uses graph neural networks to detect malicious traffic. In: 2022 4th international conference on frontiers technology of information and computer (ICFTIC). IEEE, pp 949–958. <https://doi.org/10.1109/icftic57696.2022.10075261>
- Zhang L, Shi H, Zhang K, et al (2023) GraphMAL: a network malicious traffic identification method based on graph neural network. In: 2023 international conference on networking and network applications (NaNA). IEEE, pp 262–267. <https://doi.org/10.1109/nana60121.2023.00051>
- Zheng L, Wei Z (2024) A survey of dynamic graph neural networks. *Front Comp Sci* 19(6):196323
- Zhong M, Lin M, Zhang C et al (2024) A survey on graph neural networks for intrusion detection systems: methods, trends and challenges. *Comput Secur* 141:103821. <https://doi.org/10.1016/j.cose.2024.103821>
- Zhou J, Xu Z, Rush AM, Yu M (2020) Automating botnet detection with graph neural networks. [arXiv:2003.06344](https://arxiv.org/abs/2003.06344)
- Zhou G, Liu Z, Fu C, et al (2023) An efficient design of intelligent network data plane. In: 32nd USENIX security symposium (USENIX Security 23). USENIX Association, Anaheim, CA, pp 6203–6220. <https://www.usenix.org/conference/usenixsecurity23/presentation/zhou-guangmeng>
- Zola F, Seguro-Gil L, Bruse J et al (2022) Network traffic analysis through node behaviour classification: a graph-based approach with temporal dissection and data-level preprocessing. *Comput Secur* 115:102632. <https://doi.org/10.1016/j.cose.2022.102632>
- Zoubir A, Missaoui B (2025) Graph neural networks with scattering transform for network anomaly detection. *Eng Appl Artif Intell* 150:110546. <https://doi.org/10.1016/j.engappai.2025.110546>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Mohammad Reza Ismaeli Taba¹ · Hadi Zare¹  · Saeid Rakhshani¹

✉ Hadi Zare
h.zare@ut.ac.ir

Mohammad Reza Ismaeli Taba
ismaelitaba@ut.ac.ir

Saeid Rakhshani
saeed.rakhshani@ut.ac.ir

¹ Department of Data Science and Technology, School of Intelligent Systems Engineering, University of Tehran, Tehran, Iran